



哈爾濱工業大學 (深圳)
HARBIN INSTITUTE OF TECHNOLOGY, SHENZHEN

操作系统 (Operating System)

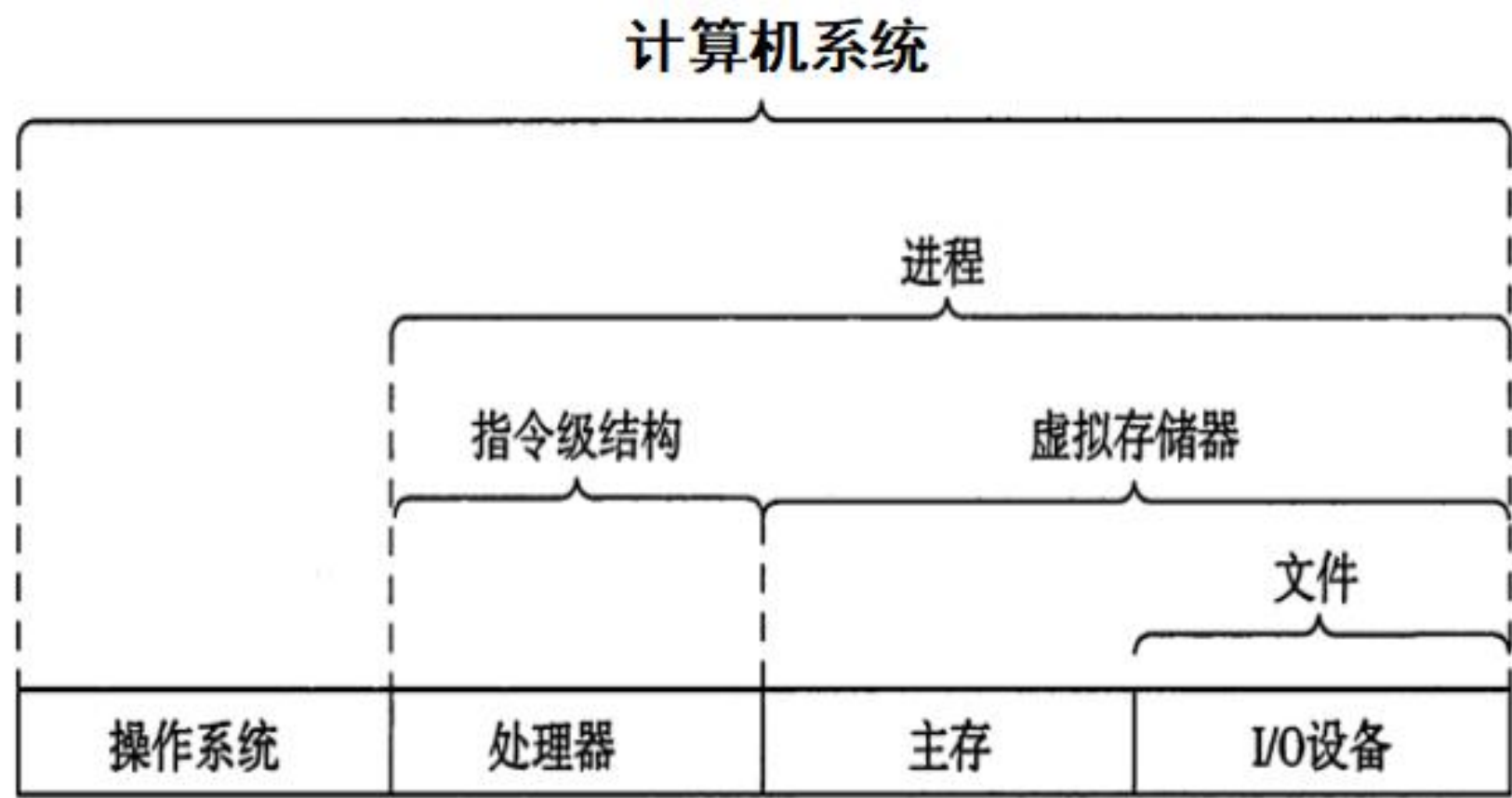
第六章：内存管理

陈芳林 副教授

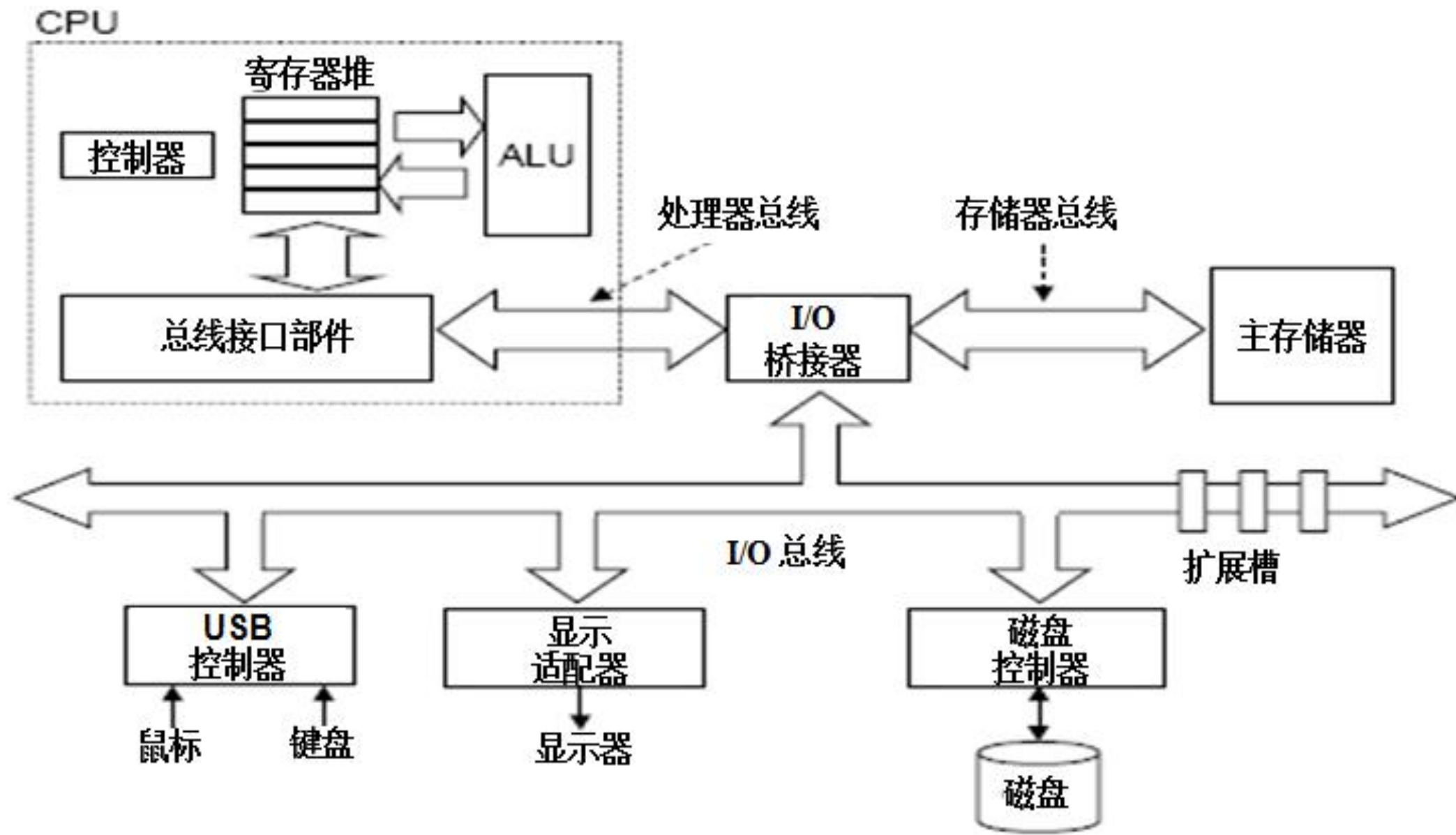
哈尔滨工业大学（深圳）

2024年秋

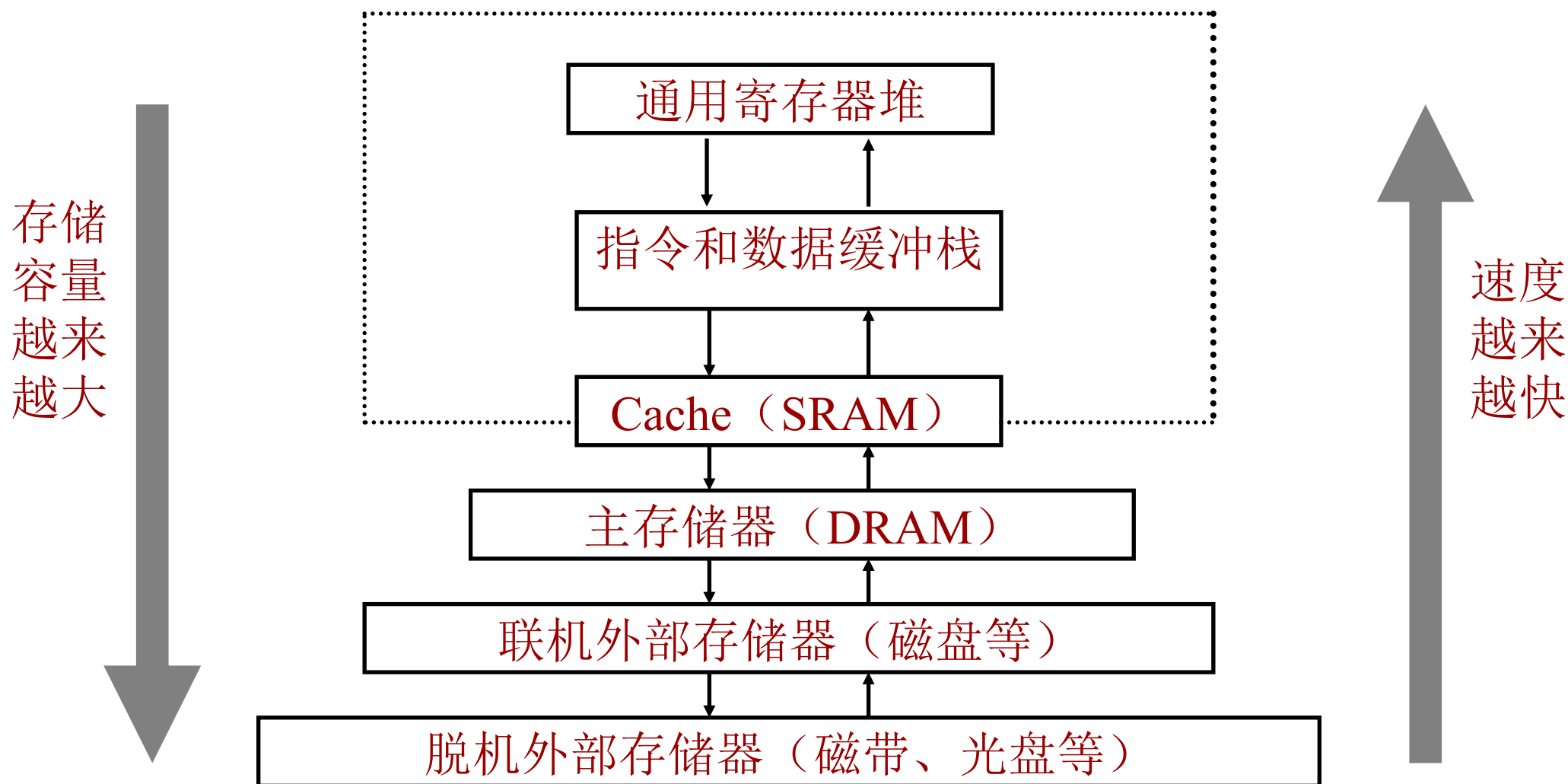
Email: chenfanglin@hit.edu.cn



- 1、导论与操作系统结构
- 2、进程与线程
- 3、并发与同步
- 4、处理器调度
- 5、死锁
- 6、内存管理**
- 7、虚拟内存
- 8、I/O与存储
- 9、文件及文件系统



■ 存储与计算的瓶颈问题



内存地址问题？

```
(gdb) l
1      #include <stdio.h>
2
3      void foo() {
4          int i = 0;
5          printf("%d\n", i);
6      }
7
8      int main(int argc, char *argv[])
9      {
10         foo();
(gdb) break 5
Breakpoint 1 at 0x659: file test.c, line 5.
(gdb) r
Starting program: /home/yjs/tmp/a.out

Breakpoint 1, foo () at test.c:5
5          printf("%d\n", i);
(gdb) p i
$1 = 0
(gdb) p &i
$2 = (int *) 0x7fffffffef4cc
```

- 这个值对应多大的内存地址空间？
- 内存不够怎么办？
- 再运行一遍，地址会不会发生变化？

■ 1 背景

- (1) 程序的装入、运行和地址映射;
- (2) 逻辑地址空间和物理地址空间
- (3) 进程内外存交换

■ 2 连续内存分配

- (1) 固定等长分区、固定变长分区、可变分区、分区分配算法;
- (2) 碎片问题与内存紧缩

■ 3 分段内存管理

■ 4 分页内存管理

■ 5 段页结合内存管理

■ 1 背景

- (1) 程序的装入、运行和地址映射;
- (2) 逻辑地址空间和物理地址空间
- (3) 进程内外存交换

■ 2 连续内存分配

- (1) 固定等长分区、固定变长分区、可变分区、分区分配算法;
- (2) 碎片问题与内存紧缩

■ 3 分段内存管理

■ 4 分页内存管理

■ 5 段页结合内存管理

- 内存是做什么用的？
- 程序是如何装入内存的？
- 数据在内存中是如何组织的？
- 程序在运行中地址是如何管理的？
- 内存不够用怎么办？

■ 执行程序是计算机的基本任务

```
int main(int argc, char* argv[])
{
    int i, to, sum = 0;
    to = atoi(argv[1]);
    for(i=1; i<=to; i++)
    {
        sum = sum + i;
    }
    printf("%d", sum);
}
```



```
C:\>sum 3127
4890628
C:\>sum 6656
22154496
C:\>sum 12345
76205685
C:\>sum 32178
517727931
C:\>_
```

■ 多个源文件怎么办? → 静态链接 or 动态链接

让程序执行起来就成了最重要的事!

■ 第一步: 编译——从C到汇编

```
int main(int argc, char*  
argv[])  
{  
    int i, to, sum =  
0;  
    to = atoi(argv[1]);  
    for(i=1; i<=to; i++)  
    {  
        sum = sum + i;  
    }  
    printf("%d", sum);  
}
```

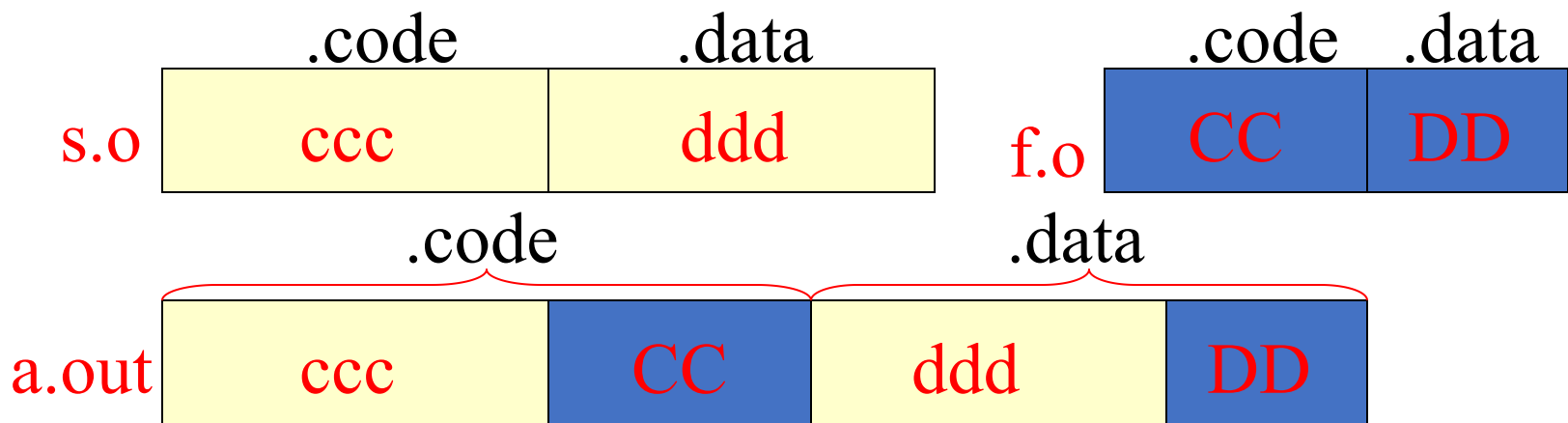
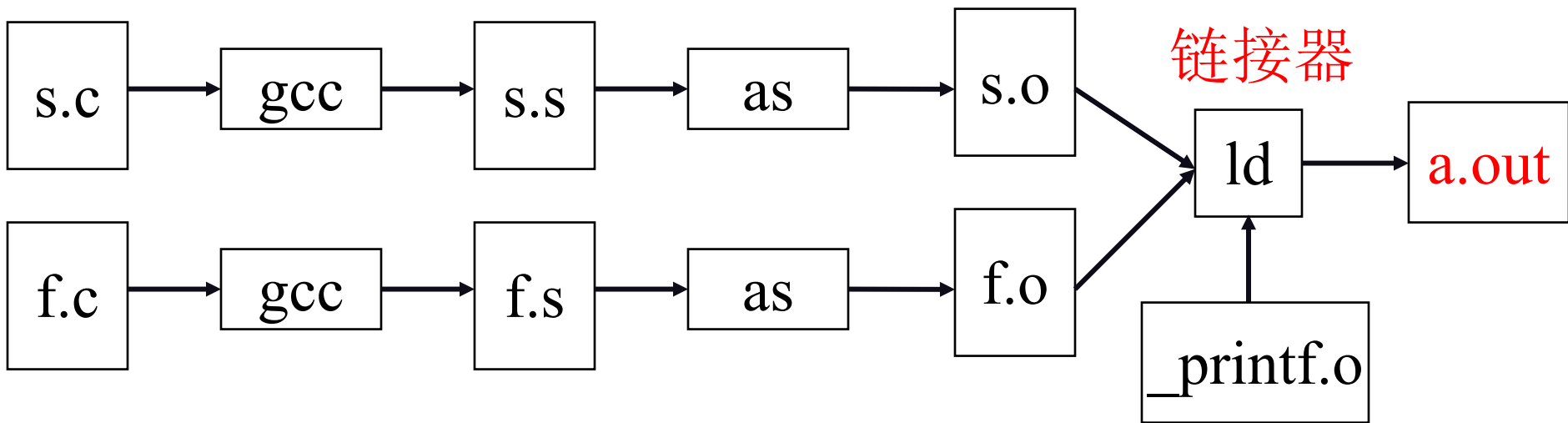
源代码

```
.text  
_entry: //入口地址  
    mov ax, [8+sp]  
    mov [_environ], ax  
    call _main  
    push ax //main返回值  
    call _exit  
_main:  
    mov [_sum], 0  
    sub sp, 4  
    mov [sp+4],  
[_environ+4]  
    call _atoi  
    mov [_to], ax //atoi返回ax  
    mov [_i], 1
```

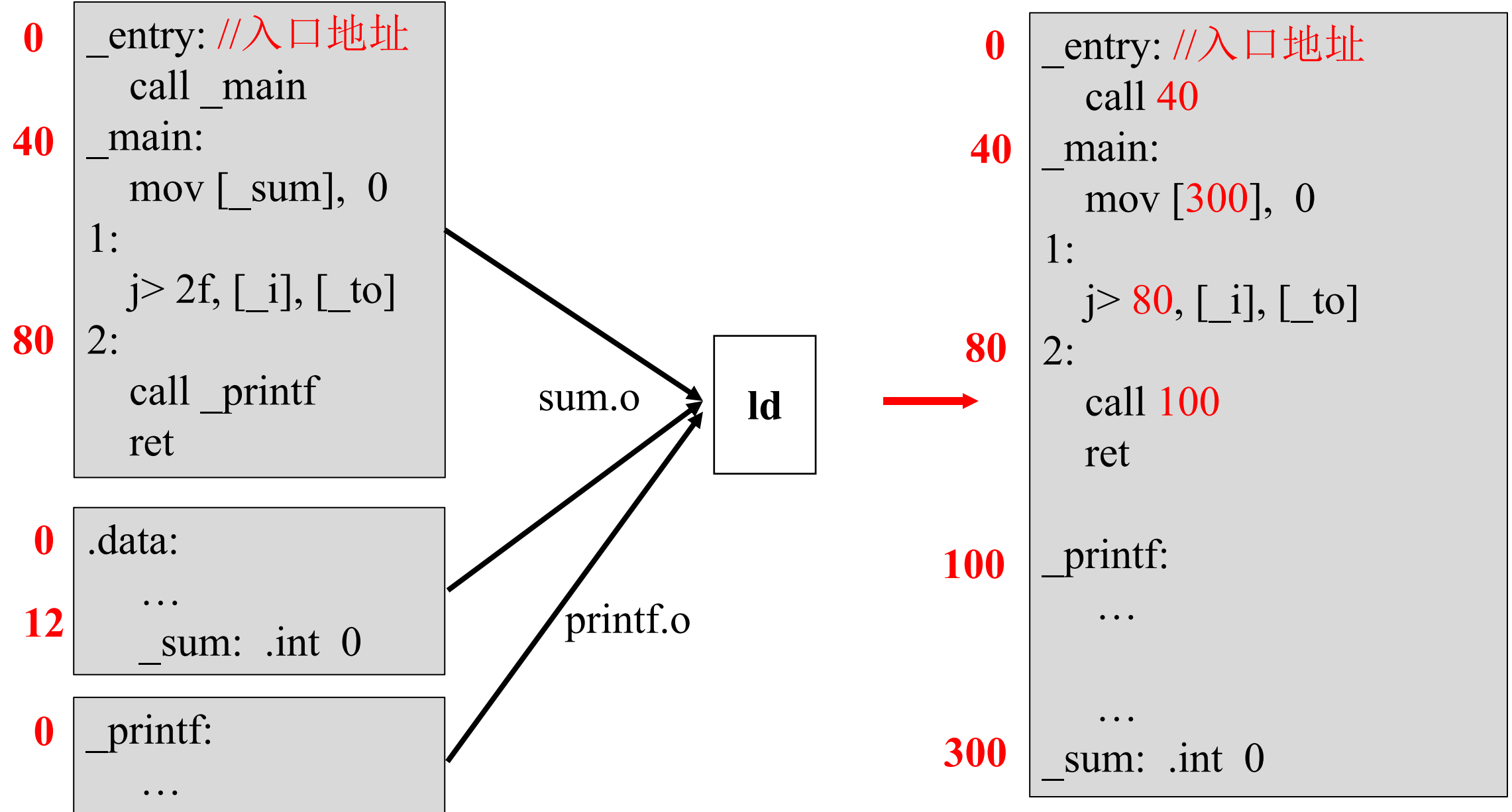
汇编代码

```
1:  
    jgt 2f, [_i], [_to]  
    mov ax, [_sum]  
    add ax, [_i]  
    mov [_sum], ax  
    add [_i], 1  
    jmp 1b  
2: sub sp, 8  
    mov [sp+4], [_sum]  
    call _printf  
    ret  
.data:  
    _environ: .long 0  
    _i: .int 0  
    _to: .int 0  
    _sum: .int 0
```

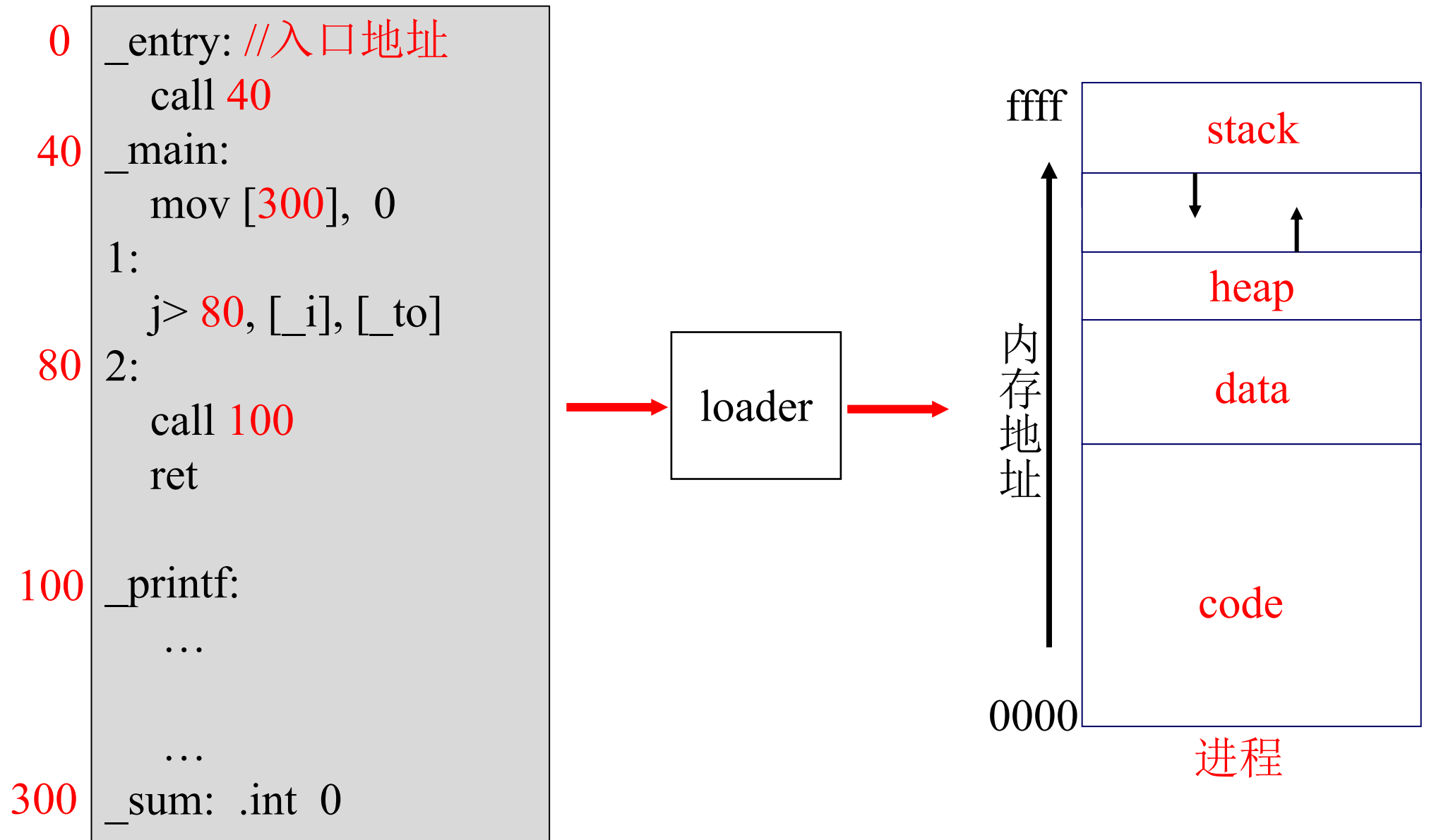
■ 第二步: 汇编与链接——从汇编到可执行程序



前面的程序经过链接以后...



现在还是程序，不是进程



程序可以执行了吗?

1000		stack
304		heap
300	_sum: .int 0	
288	...	data
	...	
100	_printf:	code
	...	
	ret	
80	2:call 100	
	1: j> 80, [_i], [_to]	
40	_main:mov [300], 0	
0	call 40	

- 程序怎么能正确开始?
- 将代码段放在内存中从0开始的地方
- 将数据段放在内存中从288开始的地方
- 设置PC=0
- 如果内存中从0开始的一段内存有专门的用途怎么办? 例如: 存放中断处理程序



需要重定位!

1000是由硬件和操作系统决定的!

■ 假设内存从地址1000以后是可以使用的

```
1300  _sum: .int 0
1288  ...
      ...
1100  _printf:
      ...
      ret
1080  2:call 100
      1: j> 80, [_i], [_to]
1040  _main:mov [300], 0
1000  call 40
```

重定位



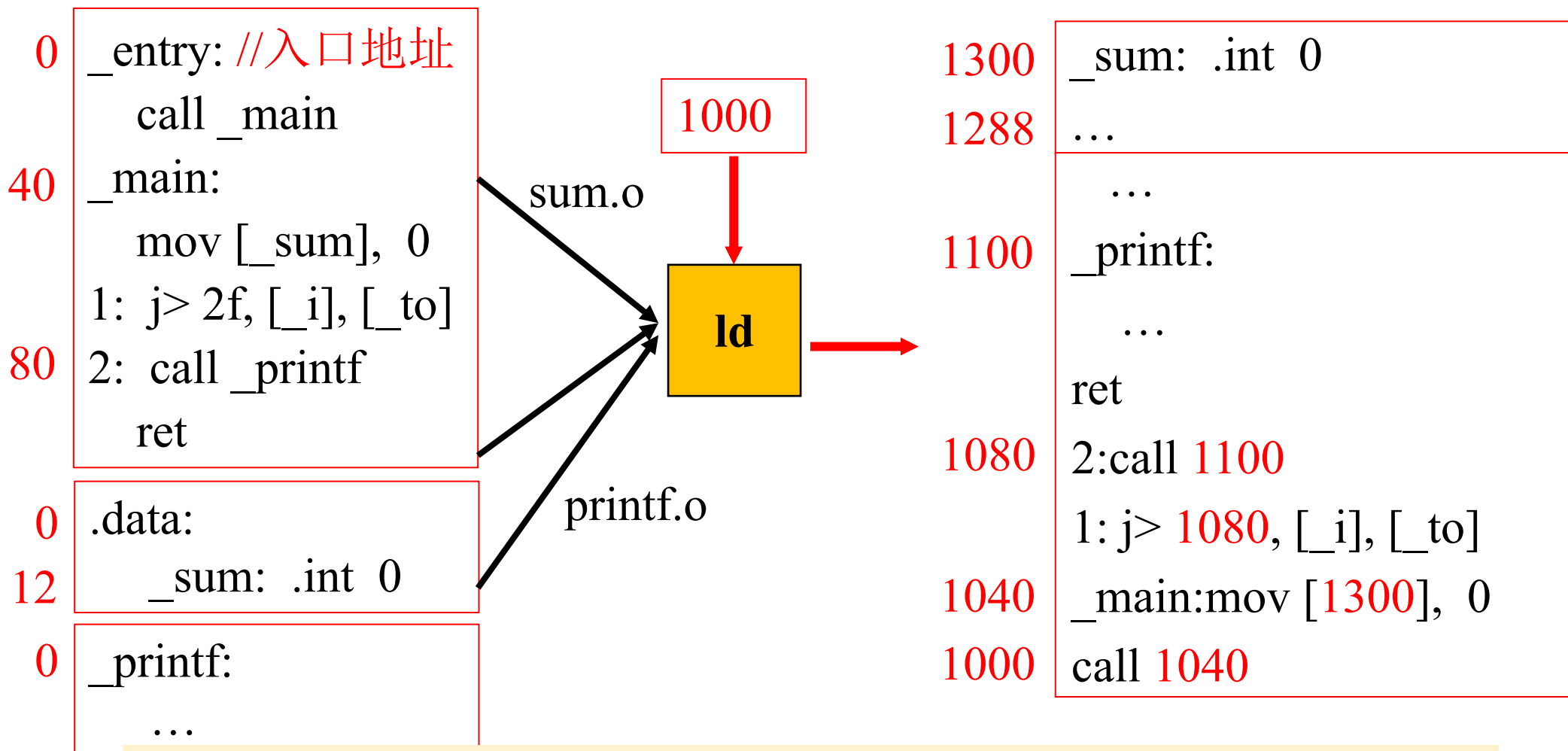
```
1300  _sum: .int 0
1288  ...
      ...
1100  _printf:
      ...
      ret
1080  2:call 1100
      1: j> 1080, [_i], [_to]
1040  _main:mov [1300], 0
1000  call 1040
```

重定位: 为执行程序而对其中出现的地址所做的修改

重定位可以执行的时机

这样的代码叫
绝对代码!

■ 第一种时机: 在编译链接时

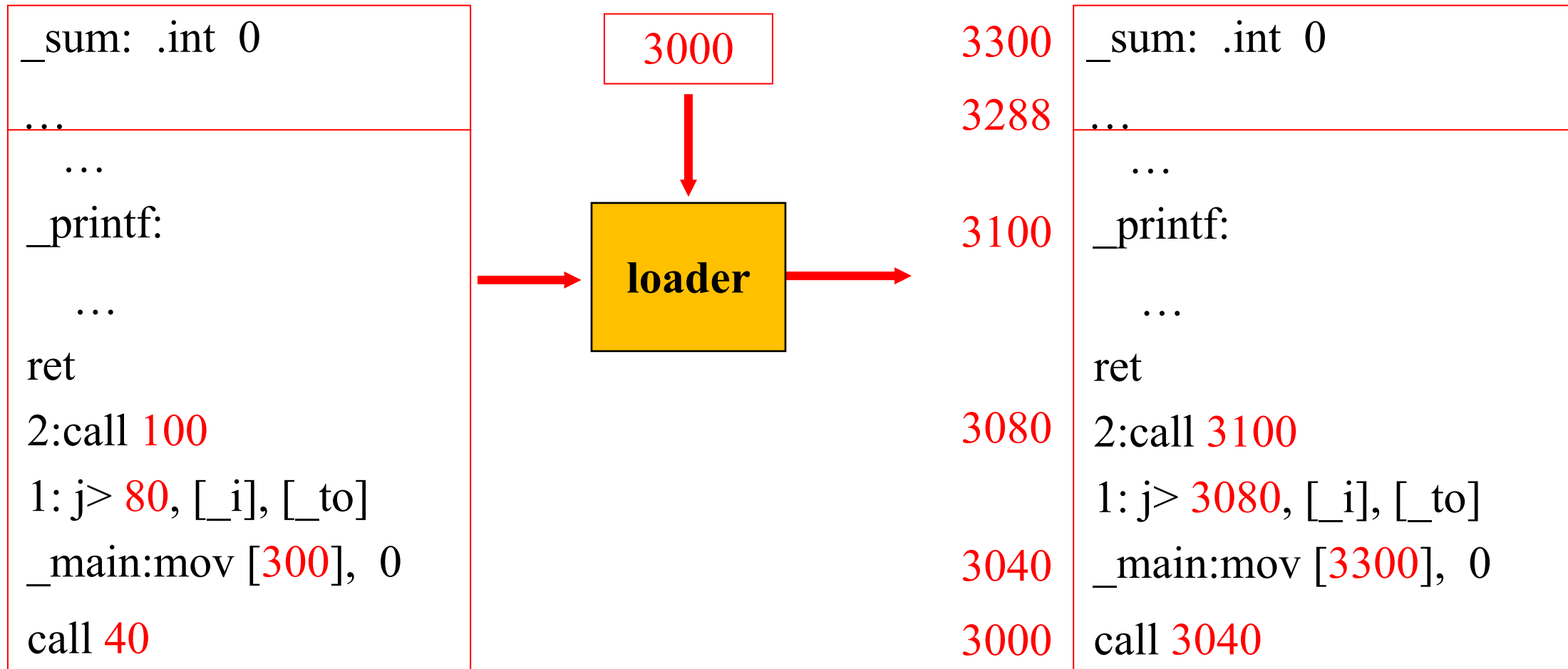


绝对代码: 这样的代码只能放在事先确定的位置上

并发 => 多个程序同时在内存中

可重定位代码!

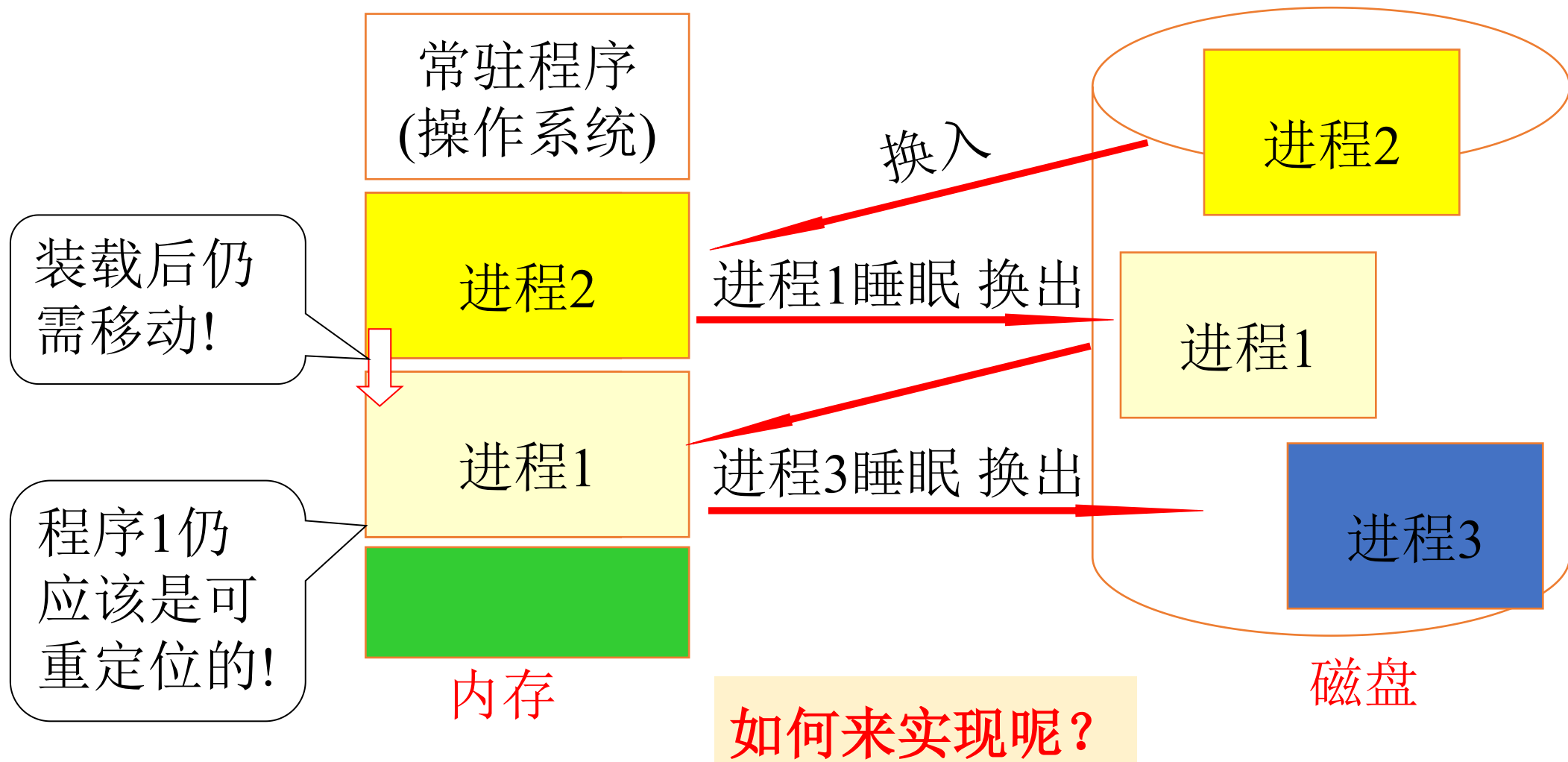
■ 分别用1000, 2000, ... 吗? 第二种时机: 载入时



装载时的重定位仍然存在缺点...一旦载入不能移动

移动也是很有必要的!

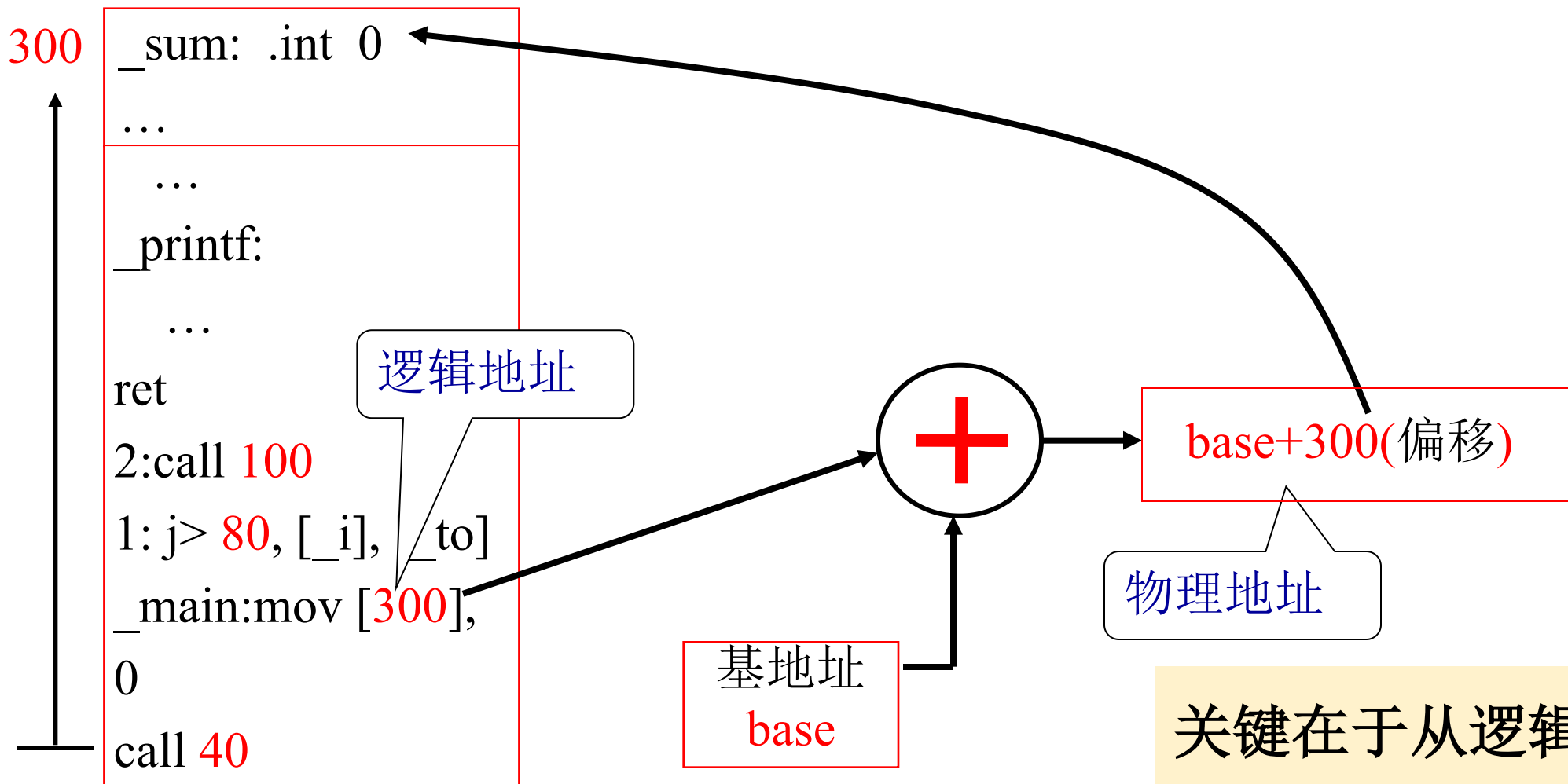
- 一个重要概念: 交换(swap)
- 能让更多的进程并发, 提高内存的利用率



重定位最合适的时机--运行时重定位

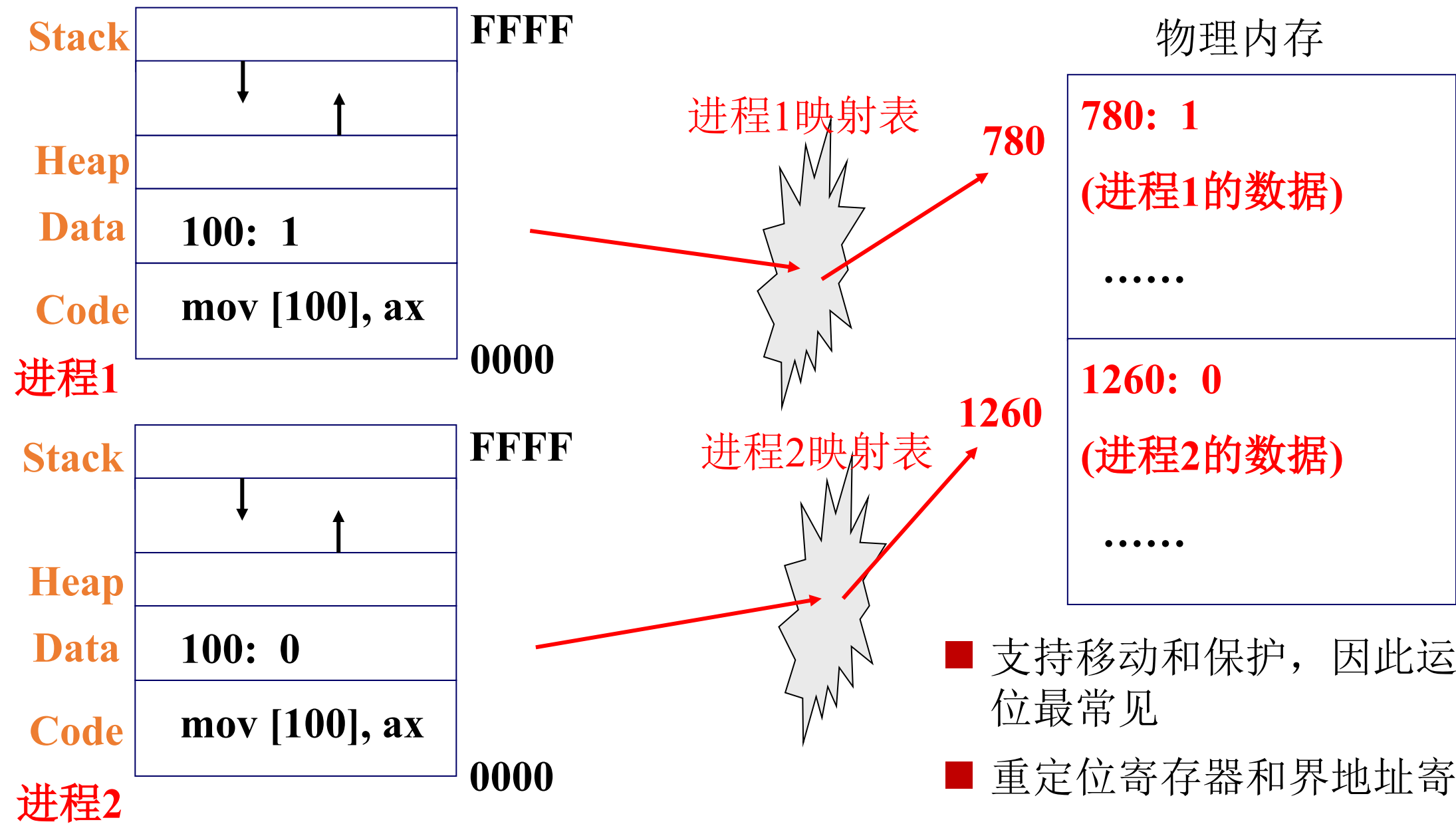


■ 内存中的代码总是可重定位的!



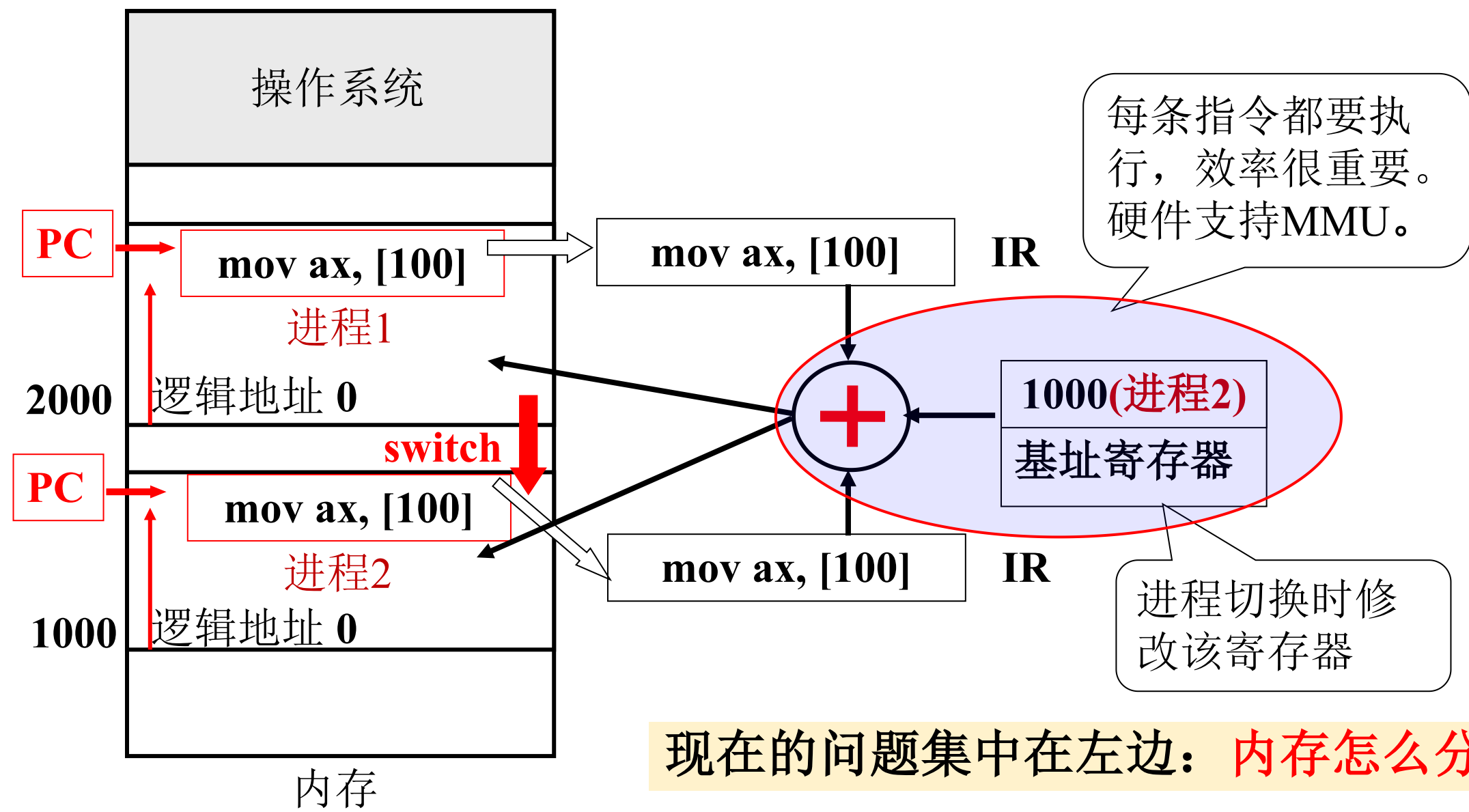
关键在于从逻辑地址算出物理地址：**地址翻译**

运行时重定位还有一个好处：进程保护



- 支持移动和保护，因此运行时重定位最常见
- 重定位寄存器和基址寄存器

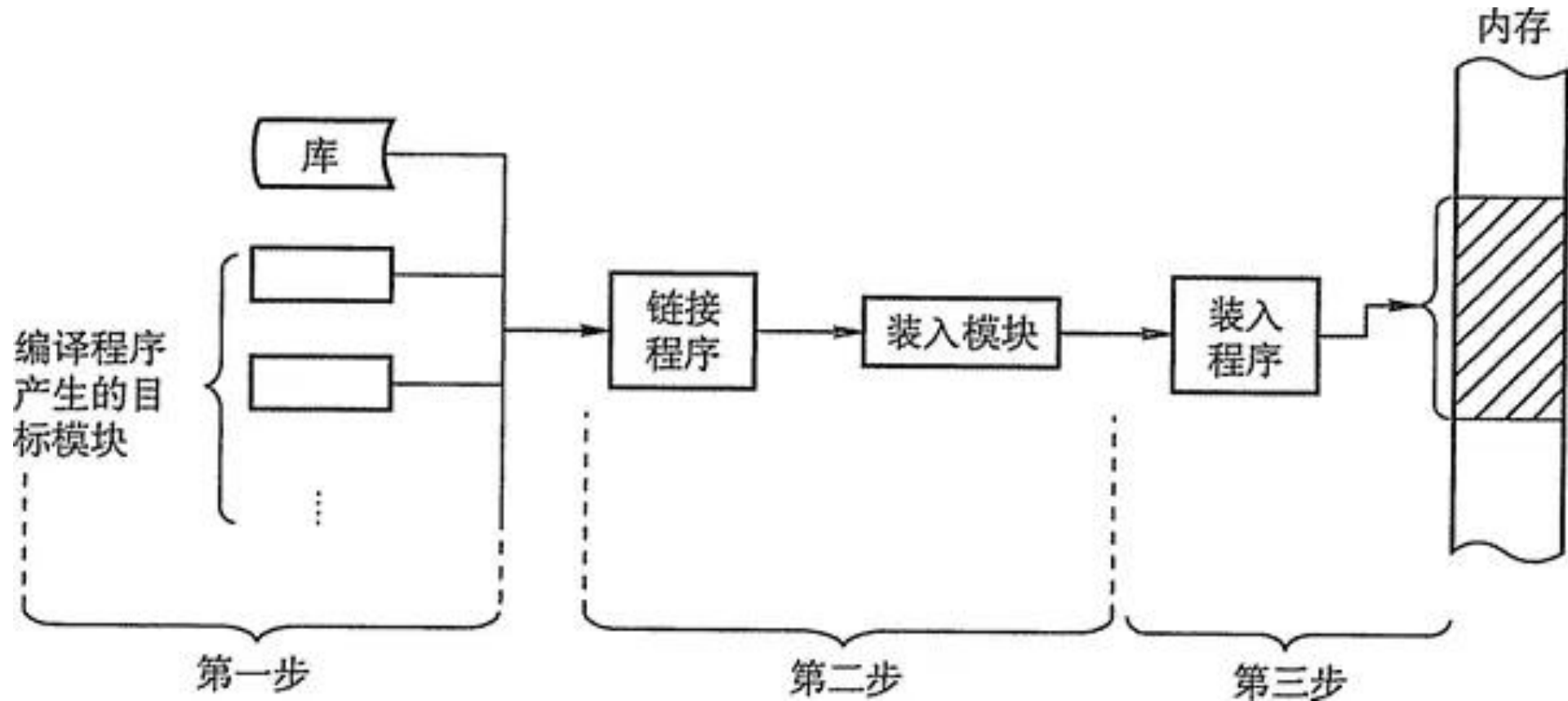
整理一下思路...



- 操作系统将可执行程序加载到内存，合理分配内存。
- 可执行程序经过编译产生了逻辑地址，为了提高进程的并发和内存利用率，逻辑地址和运行物理地址间需要地址转换。
- 嵌入式操作系统多数内存管理非常简单，甚至没有内存管理。
- PC和服务端、分布式等现代操作系统基本都支持虚拟内存管理。

简单总结一下（续）

- 创建进程首先要将程序和数据装入内存。将用户源程序变为可在内存中执行的程序，通常需要**编译**、**链接**、**装入**三个步骤



- 虚拟地址：用户编程时将代码（或数据）分成若干个段，每条代码或每个数据的地址由段名称 + 段内相对地址构成，这样的程序地址称为虚拟地址
- 逻辑地址：虚拟地址中，段内相对地址部分称为逻辑地址
- 物理地址：实际物理内存中所看到的存储地址称为物理地址
- 比较：虚拟地址由用户编写程序时定义的**全局地址**；逻辑地址是用户定义的**局部地址**，是虚拟地址的组成部分；物理地址就是实际存在的以Byte为单位的**存储单元的编号**

■ 逻辑地址空间：

在实际应用中，将虚拟地址和逻辑地址经常不加区分，**统称为逻辑地址**。逻辑地址的集合称为逻辑地址空间

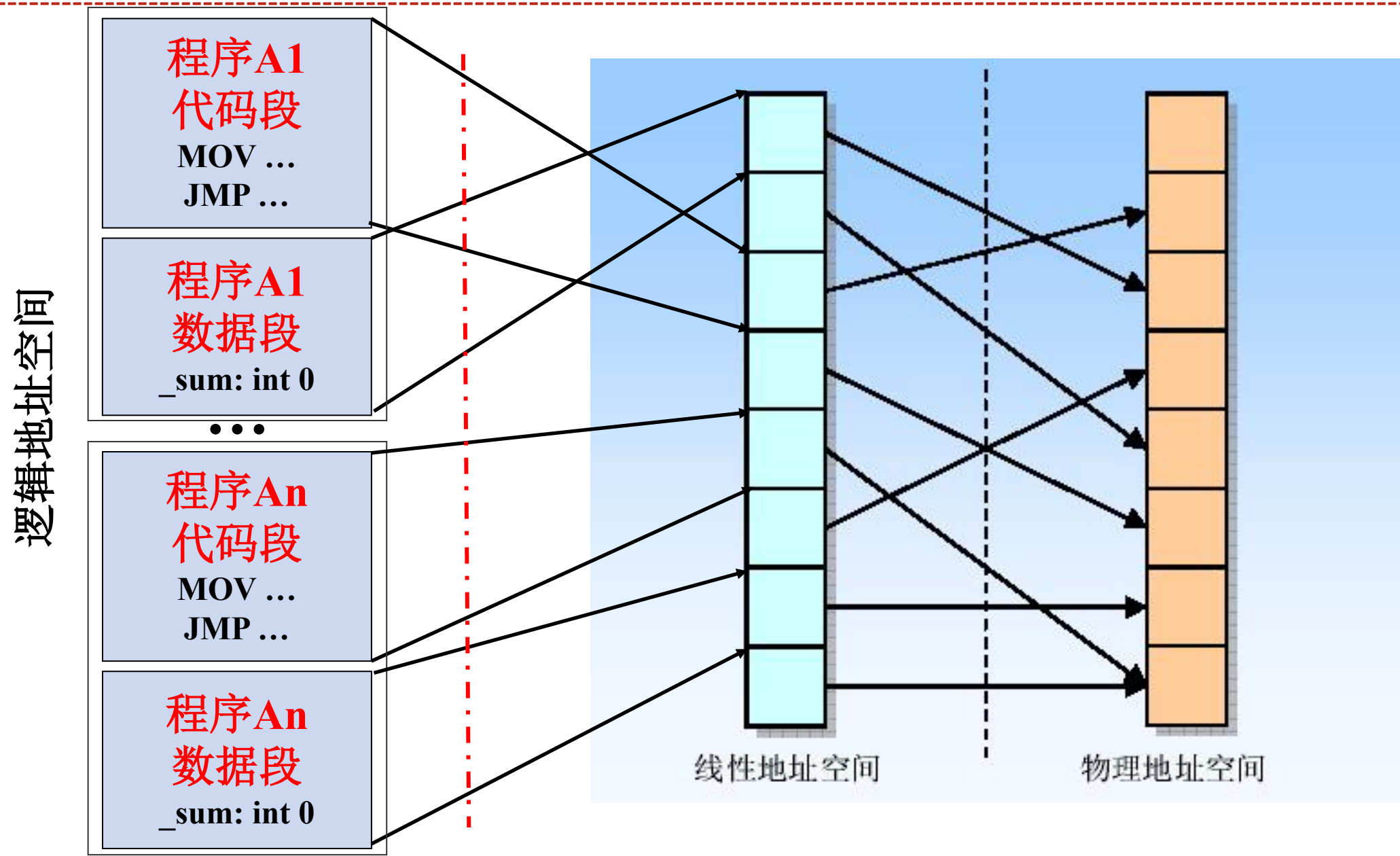
■ 线性地址空间：

CPU地址总线可以访问的所有地址集合称为线性地址空间

■ 物理地址空间：

实际存在的可访问的物理内存地址集合称为物理地址空间

逻辑地址空间-线性地址空间-物理地址空间的关系



■ MMU (Memory Management Unit 内存管理单元) :

实现将用户程序的虚拟地址 (逻辑地址) → 物理地址映射的CPU中的硬件电路

■ 基地址:

在进行地址映射时, 经常以段或页为单位并以其最小地址 (即起始地址) 为基值来进行计算

■ 偏移量:

在以段或页为单位进行地址映射时, 相对于基地址的地址值



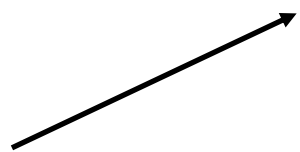
虚拟地址与物理地址

■ 嵌入式操作系统多数内存管理非常简单，甚至没有内存管理。虚拟地址=物理地址

■ PC和服务端、分布式等现代操作系统基本都支持虚拟内存管理。虚拟地址!=物理地址



■ 前者链接时重定位



■ 后者运行时重定位

- 内存是做什么用的
- 程序是如何装入内存的
- 数据在内存中是如何组织的
- 程序在运行中地址是如何管理的
- 内存不够用怎么办

- 连续内存分配与管理
- 分段分配与管理
- 分页分配与管理
- 段页结合管理
- 请求式分页管理（虚拟内存）

■ 1 背景

- (1) 程序的装入、运行和地址映射;
- (2) 逻辑地址空间和物理地址空间
- (3) 进程内外存交换

■ 2 连续内存分配

- (1) 固定等长分区、固定变长分区、可变分区、分区分配算法;
- (2) 碎片问题与内存紧缩

■ 3 分段内存管理

■ 4 分页内存管理

■ 5 段页结合内存管理

从最简单的分配方案开始!

(1) 固定分区--等长分区

■ 给你一个蛋糕， n 个孩子来吃，怎么办？

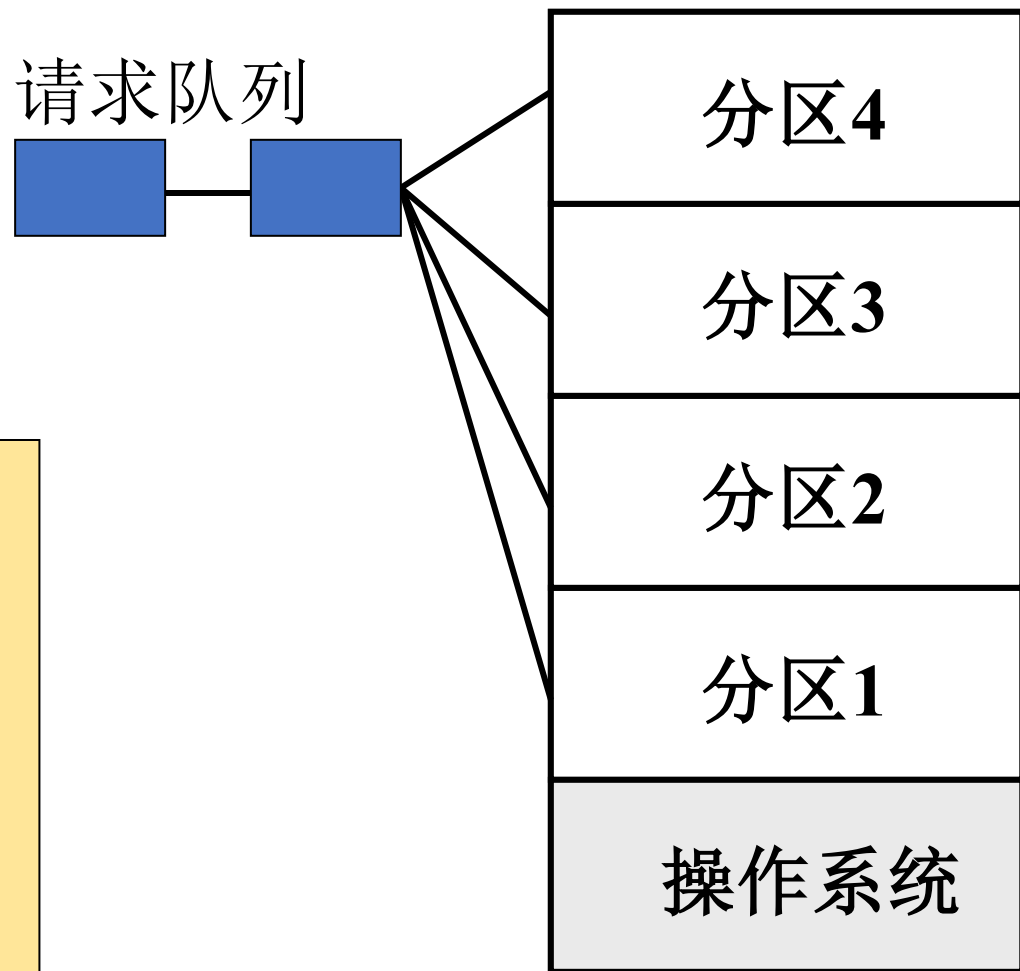
➤ 等分...

■ 操作系统初始化时将内存等分成 k 个分区

```
bool AvailSec[k]; int SecSize;
```

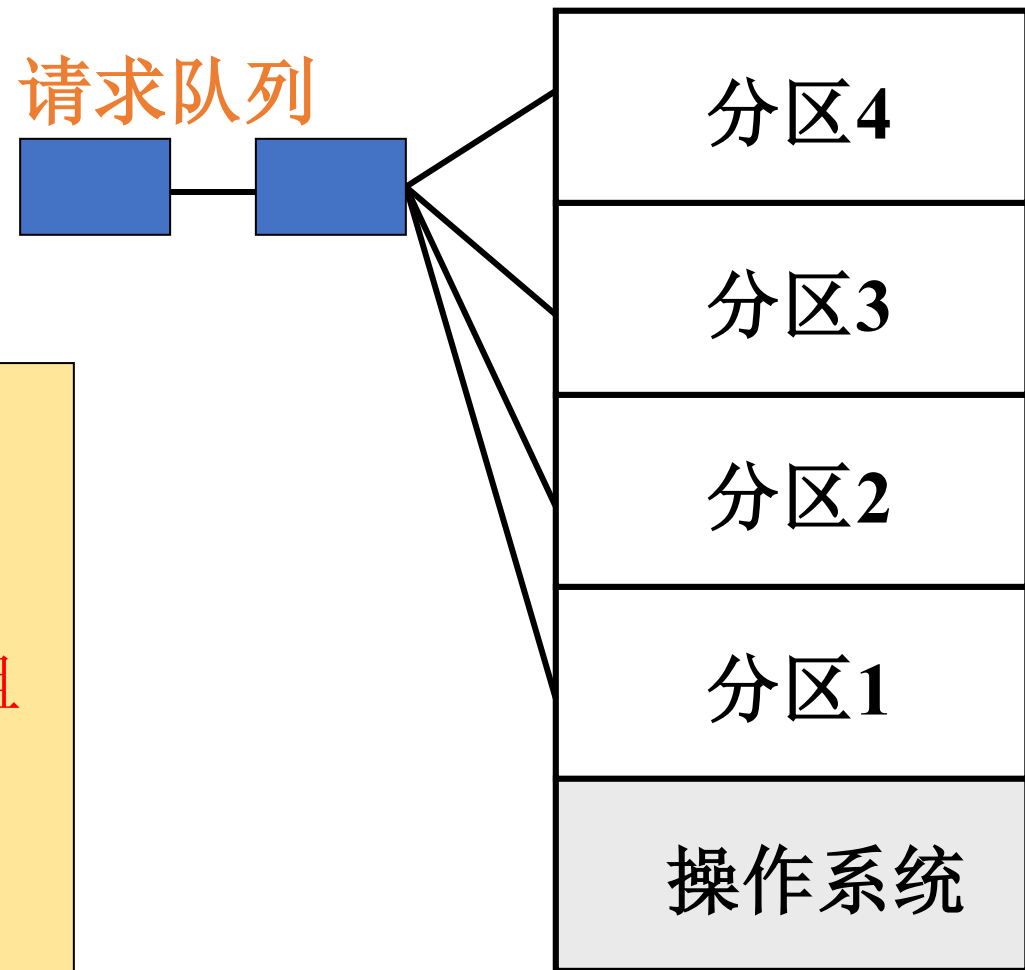
内存请求算法 //进程创建时

1. if (reqSize > SecSize) exit
2. 找出AvailSec[i]为真的 i
3. 如果有，返回分区 i 的基址
4. 否则，将current加入请求队列等待



(2)固定分区--变长分区

- 孩子有大有小，进程也有大有小...
- 初始化时将内存分成k个大小不同的分区



```
struct Section AvailSec[k];
```

内存请求算法 //进程创建时

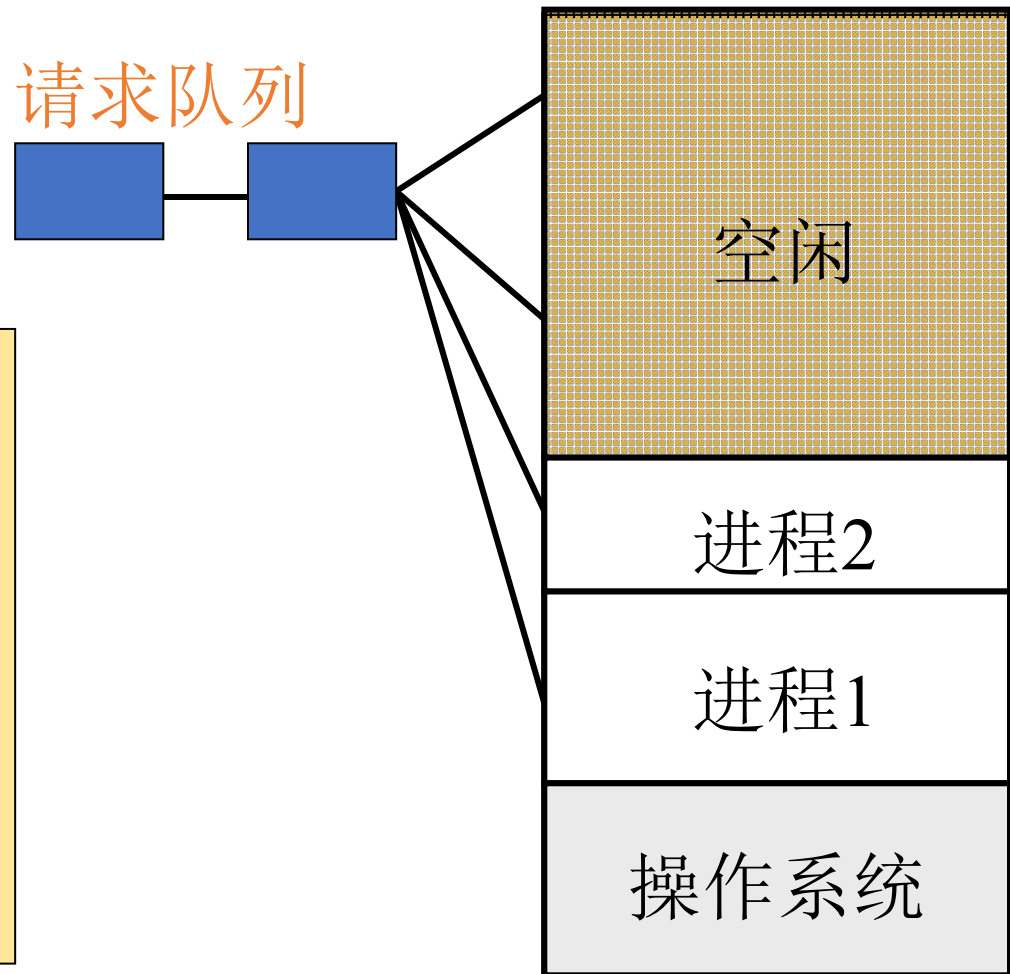
1. `if (reqSize > MaxSize) exit`
2. 找出 `AvailSec[i].Size > reqSize` 且 `AvailSec[i].Size` 最小的空闲分区 `i`
3. 如果有，返回分区 `i` 的基址
4. 否则，将 `current` 加入请求队列等待

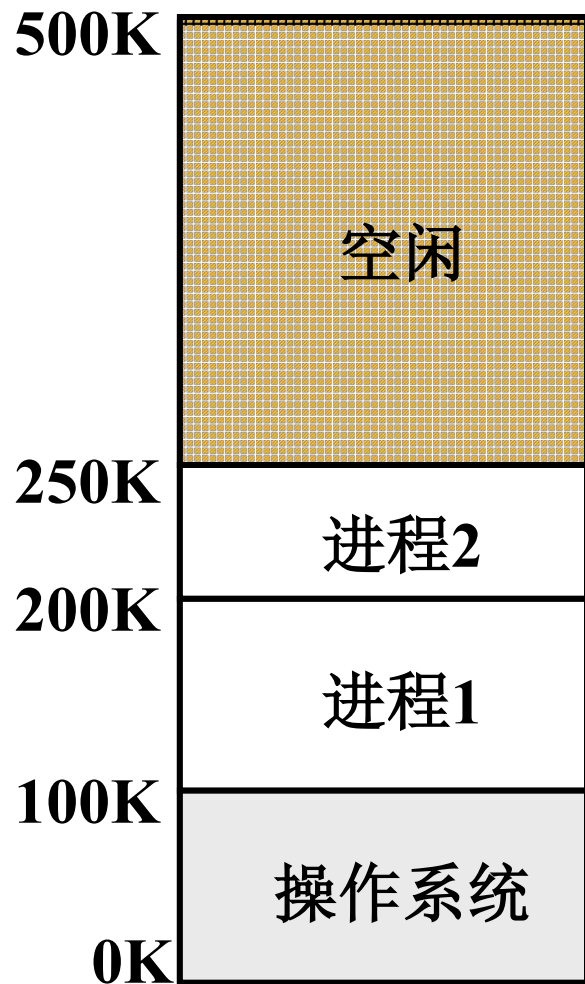
(3)可变分区

- 合理的方法应该是根据孩子饥饿程度来分割
- 根据reqSize进行动态分割

内存请求算法 //进程创建时

1. if (reqSize > 内存大小) exit
2. if (reqSize > 空闲空间总尺寸)
将current加入请求队列等待
3. 从空闲分区划出一个reqSize, 并返回
其基地址 //那个空闲分区
4. 修改分区数据结构





空闲分区表

始址	长度
250K	250K

已分配分区表

始址	长度	标志
0K	100K	OS
100K	100K	P1
200K	50K	P2

可变分区数据结构的变化(1)



- 内存请求: reqSize = 100K

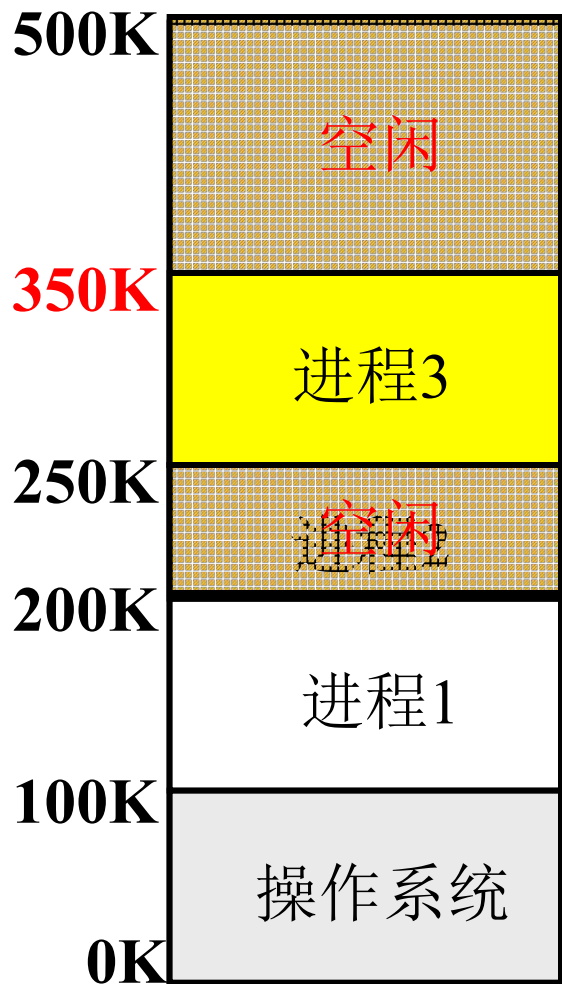
空闲分区表

始址	长度
250K	250K

已分配分区表

始址	长度	标志
0K	100K	OS
100K	100K	P1
200K	50K	P2
250K	100K	P3

可变分区数据结构的变化(2)



- 进程2执行完毕，释放内存

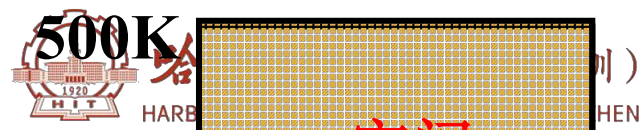
空闲分区表

始址	长度
350K	150K
200K	50K

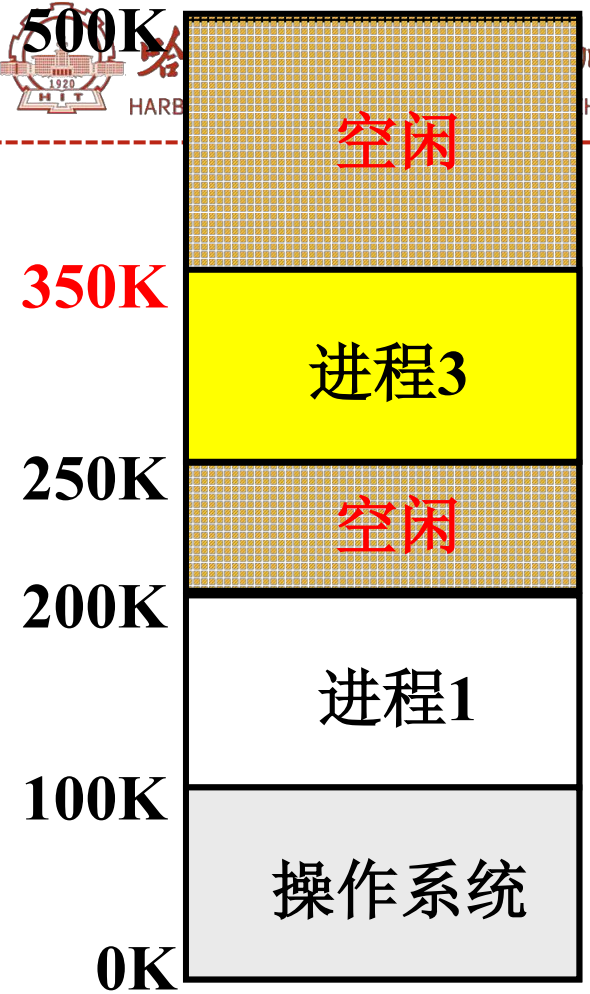
已分配分区表

始址	长度	标志
0K	100K	OS
100K	100K	P1
200K	100K	P3
250K	100K	P3

(4)分区分配算法--找到合适的空闲分区



- 发起请求reqSize=40K怎么办?
- 首次适配(first-fit):首次找到的满足要求的空闲分区(350,150)。特点: 快速!
- 最佳适配(best-fit): 查找最小的满足要求的空闲分区(200,50)。特点: (1)搜索整个空闲分区表, 慢! (2)会产生许多小的空闲分区。
- 最差适配(worst-fit):查找最大的满足要求的空闲分区(350,150)。特点: (1)搜索整个空闲分区表, 慢! (2)新产生的空闲分区大一些。



空闲分区表

始址	长度
350K	150K
200K	50K

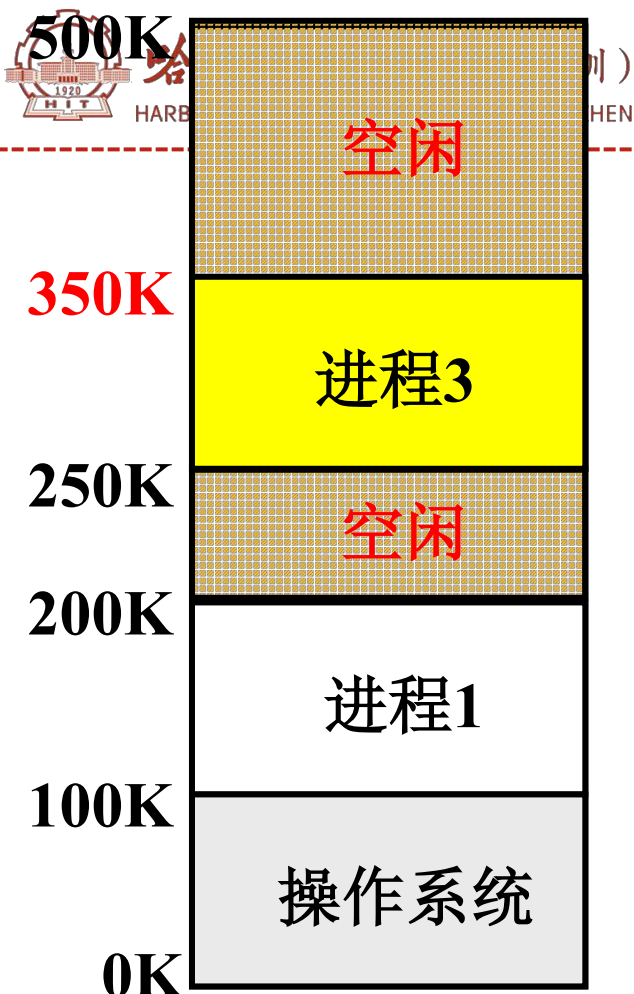
(5)碎片问题

- 碎片：内存中剩余的无法使用的存储空间
- 外部碎片：随着进程不断的装入和移出，对分区不断的分割，使得内存中产生许多特别小的分区，它们并不连续可用
- 内部碎片：对固定分区来说，只要分区被分配给某进程使用，其中并未占用的空间不能分给其他进程
- 碎片的产生和分配“制度”有关！不易避免！
- 碎片问题可以消除（内存紧缩），但时间成本太高！
- 磁盘也存在碎片问题！

(6)外部碎片处理--内存紧缩

- 发起请求reqSize=160K怎么办?
- 总空闲空间>160, 但没有一个空闲分区>160, 怎么办?
- 内存紧缩: 将空闲分区合并在一起, 需要移动进程3(复制内容)
- 内存紧缩需要花费大量时间, 如果一台1GB 内存的计算机可以每20ns复制4个字节, 它紧缩全部内存大约花费多久??

该值表明连续分配技术不合适!

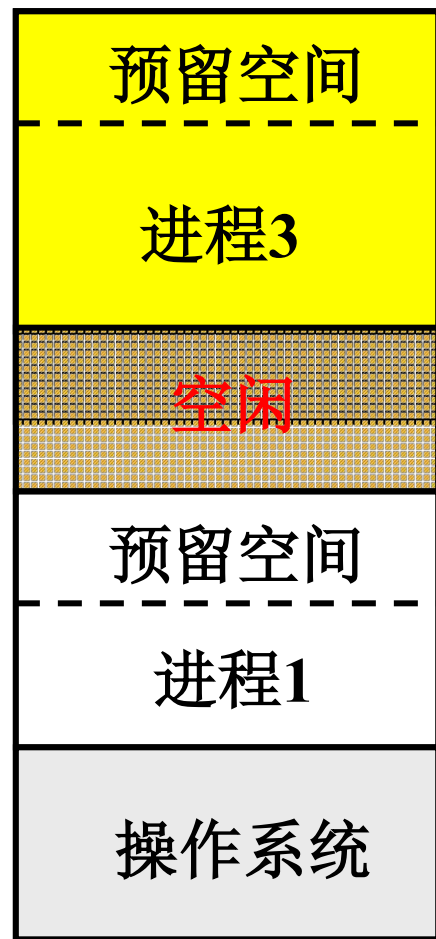


空闲分区表

始址	长度
350K	150K
200K	50K

分区方案中还有一个问题没有搞清楚

- reqSize值怎么确定?
- $\text{reqSize} = \text{code} + \text{data} + \text{stack} + \text{heap}$
- code和data不难处理, stack和heap难处理: 动态增长, 预先不知道
- 怎么办? 预留空间
- 预留空间用完了怎么办? 找一个更大的空闲空间, 移动该进程
- 实际上移动code和data纯粹是浪费。
- 各个段区别对待, 分别分配



还有利于建立合适的保护策略!

■ 1 背景

- (1) 程序的装入、运行和地址映射;
- (2) 逻辑地址空间和物理地址空间
- (3) 进程内外存交换

■ 2 连续内存分配

- (1) 固定等长分区、固定变长分区、可变分区、分区分配算法;
- (2) 碎片问题与内存紧缩

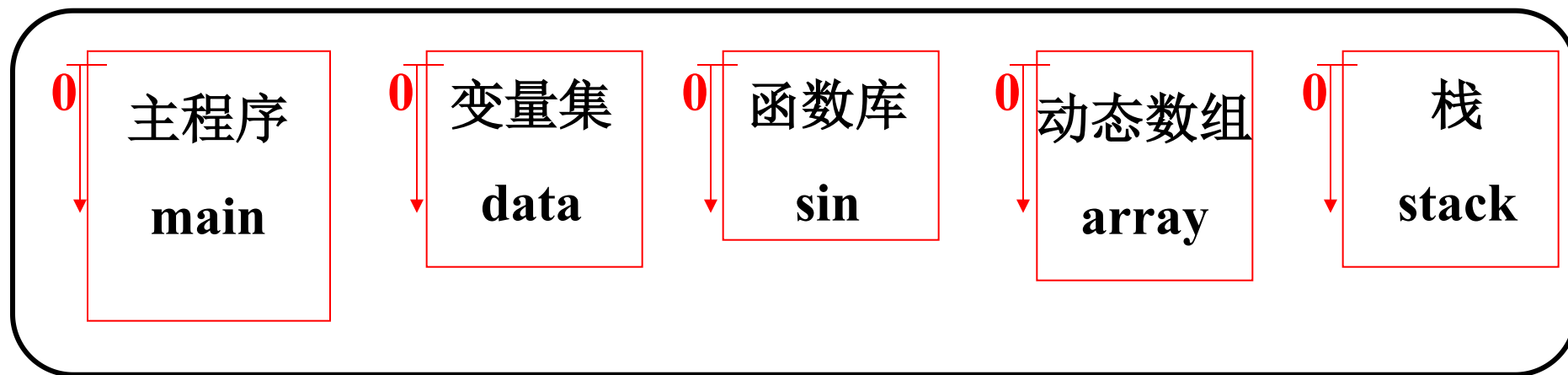
■ 3 分段内存管理

■ 4 分页内存管理

■ 5 段页结合内存管理

分段(Segmentation)!

- 由若干部分（段）组成，每个段有各自的特点、用途！

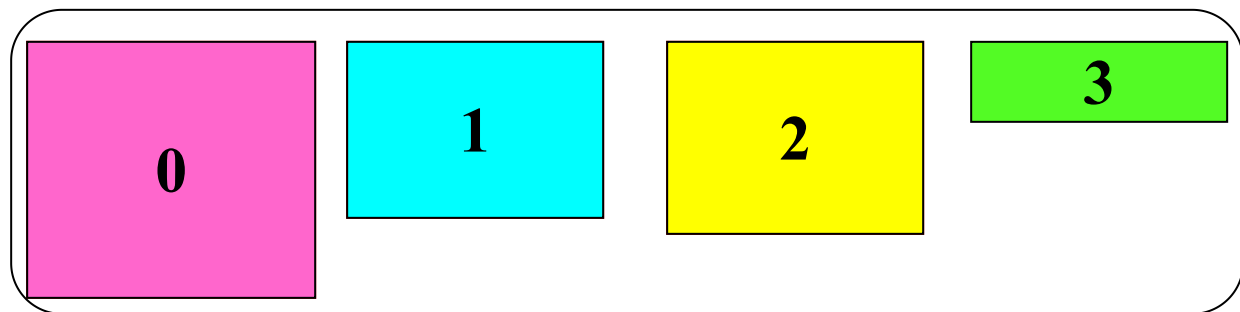


程序员眼中的一个程序

- 程序员怎么定位具体指令（数据）：**<段号, 段内偏移>**
- 分段符合用户观点：**用户可独立考虑每个段（分治）**

将段放入内存

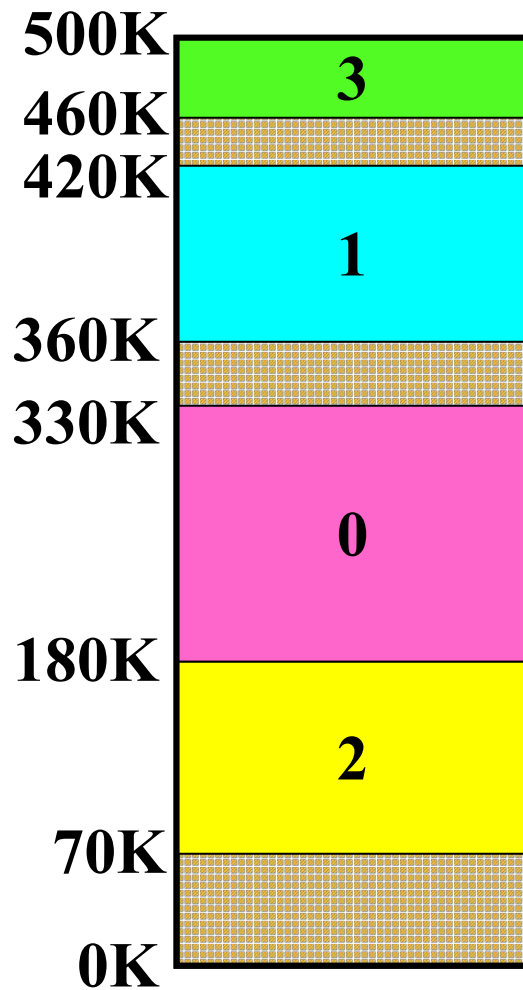
■ 分段制造了二维空间，而内存是一维的



■ 一个进程需要记录多个基址...

进程段表	段号	基址	长度	保护
	0	180K	150K	R
	1	360K	60K	R/W
	2	70K	110K	R/W
	3	460K	40K	R

各个段可分散

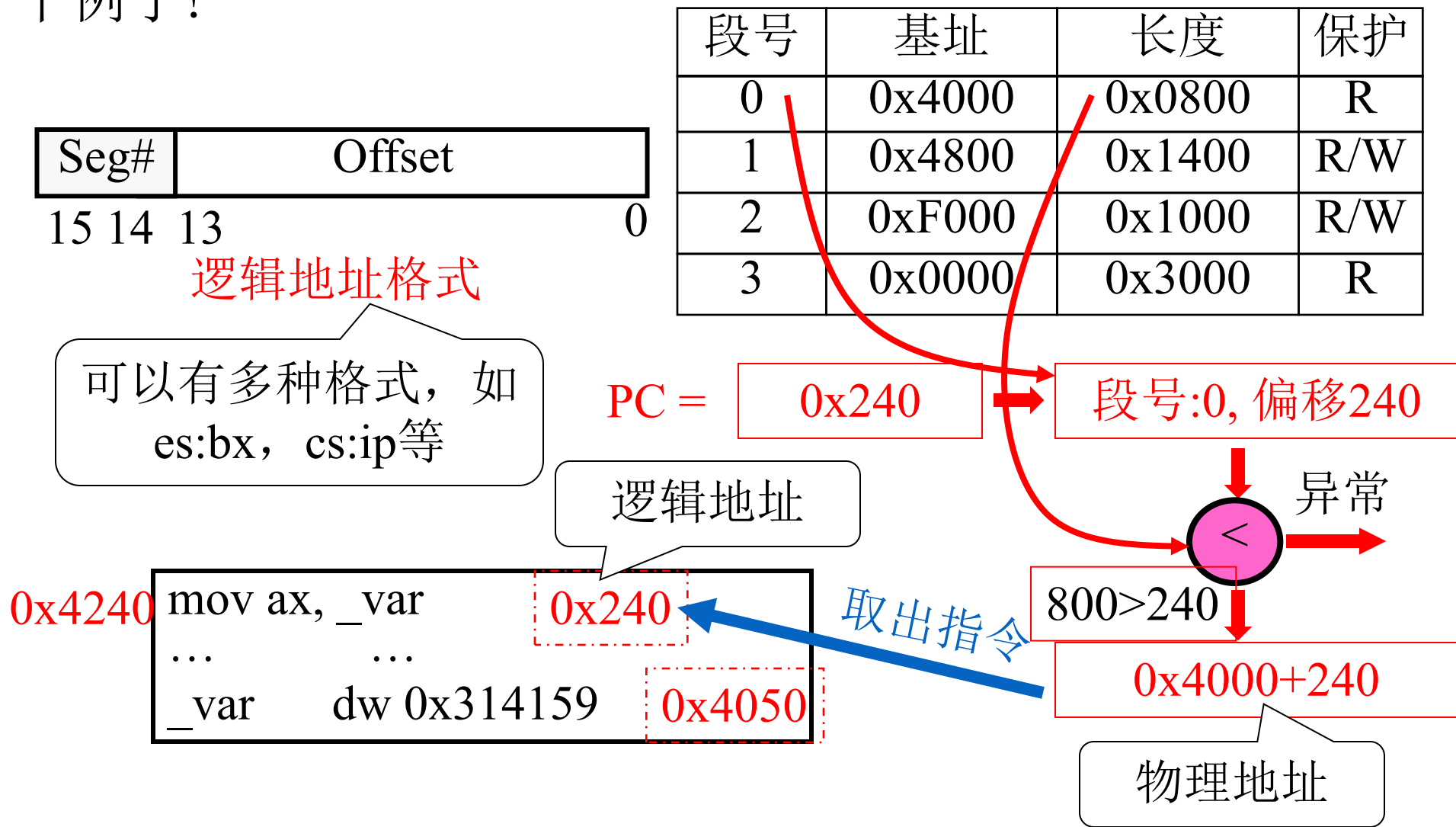


引入
段表

■ 仍有内存分区表，内存分配算法等等

分段的地址翻译

■ 来看一个例子!



段地址映射计算方法

■ 段号S和段内偏移O的计算公式

- $S = \text{INT}[A/L]$ INT: 整除函数
- $O = [A] \% L$ %: 取余
- A: 逻辑地址, L: 逻辑段大小

Seg#	Offset
15	13 0

逻辑地址格式

■ 例如: 假定段大小为 2^{14}B , 地址 $A=0\text{x}240$, 则求得: $S=0$, $O=0\text{x}240$

对于地址 $A=0\text{x}4050$, 则求得: $S=?$, $O=?$

分段的地址翻译



逻辑地址格式

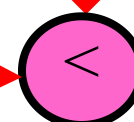
段号	基址	长度	保护
0	0x4000	0x0800	R
1	0x4800	0x1400	R/W
2	0xF000	0x1000	R/W
3	0x0000	0x3000	R

PC =

?

段号:?, 偏移?

异常



?

?

物理地址

0x4240

mov ax, _var	0x240
...	...
_var dw 0x314159	0x4050

取出指令

0x4050、0x6a50、0xdfa0对应物理地

址?

■ 实现原理

- 将程序按含义分成若干部分，即分段
- 1d从0开始编址每个段（链接速度会很快）：汇编时从0编址、链接时可不为0
- 创建进程（分别载入各个段）时，建立进程段表
- 内存仍用可变分区进行管理，载入段时需调分配算法
- PC及数据地址要通过段表算出物理地址，到达内存
- 进程切换时，进程段表也跟着切换



进程、内存、编译环境、编程思想被扭结在一起了，这正是操作系统的复杂之处！

■ 优点:

符合人的习惯，程序员感觉舒服

- 不同的段有不同的含义，可区别对待
- 每个段独立编址，编程容易（如果是一个大的一维地址空间，程序员一会儿就糊涂了！）（分治）

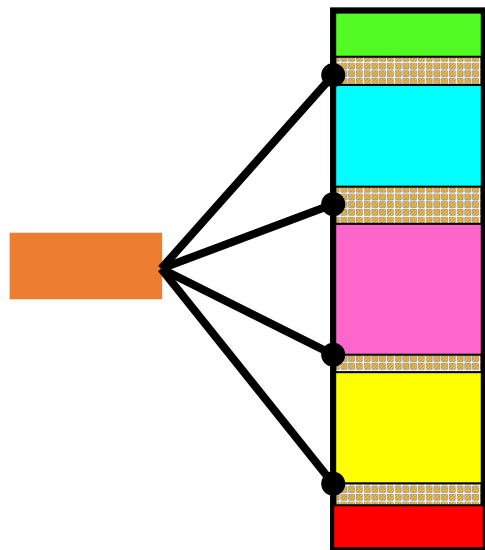
■ 缺点: 靠近了我们，必然会远离...

空间低效

- 空间预留：空闲空间很大却不能分配；内存紧缩
- 著名的碎片概念：空闲的却用不上的空间

内部碎片

外部碎片



■ 1 背景

- (1) 程序的装入、运行和地址映射;
- (2) 逻辑地址空间和物理地址空间
- (3) 进程内外存交换

■ 2 连续内存分配

- (1) 固定等长分区、固定变长分区、可变分区、分区分配算法;
- (2) 碎片问题与内存紧缩

■ 3 分段内存管理

■ 4 分页内存管理

■ 5 段页结合内存管理

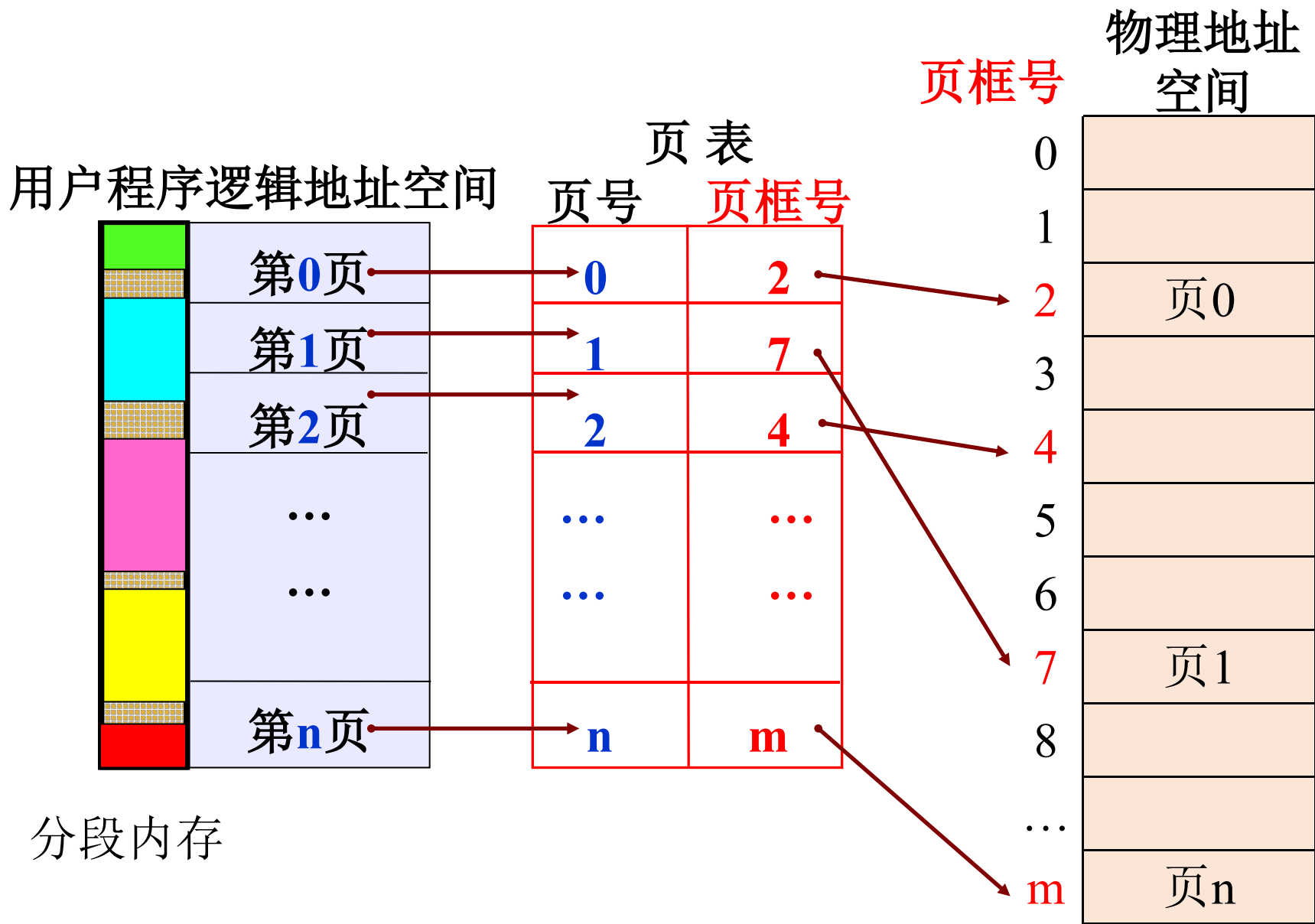
分页(Paging)!

思考：按需供给内存

从连续到离散

- 只有吃到最后才能知道到底有多饿!
- 对于进程的堆栈段，只有运行完才知道嵌套深度
- 一次分配给一点（没有外部碎片，内部碎片有上界）
- 将面包切成薄片，将内存分成页





■ 如此映射优势在哪？

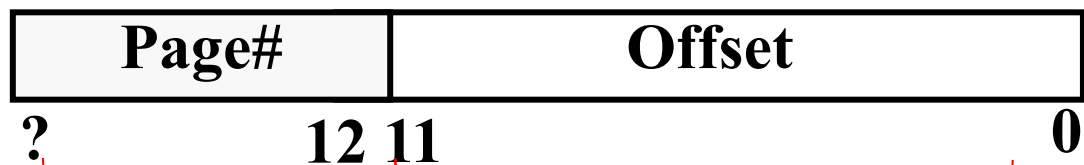
■ 1) 程序逻辑地址中的页是不是一直被访问？

■ 2) 某些未被访问的页是否映射到物理内存？

■ 3) 长时间未访问的页如何处理？

■ 和分段类似，分页依靠页表结构

逻辑地址格式



多少页

页面尺寸(4K)

进程页表

页号	页框号	保护
0	5	R
1	1	R/W
2	3	R
3	7	R

页框7

页3

页框6

页框5

页0

页框4

页框3

页2

页框2

页框1

页1

页框0

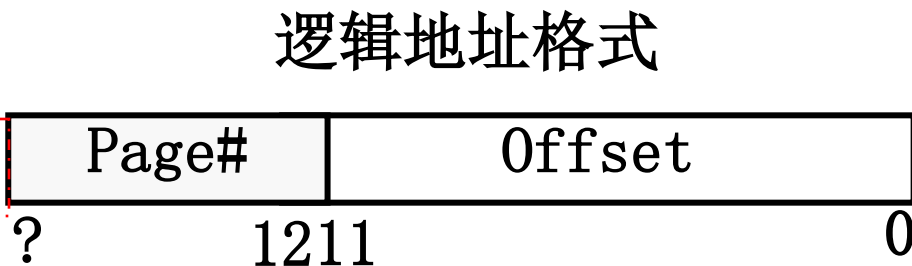
■ 一个实例!

物理地址0x79f0对应的逻辑地址?

```
mov ax, _var    0x240
...             ...
_var dw 0x314159 0x4050
```

页号

偏移



逻辑地址

0x00	0x240
------	-------

页表指针

PCB中应有此值

页号	页框号	保护
0	5	R
1	1	R/W
2	3	R/W
3	7	R

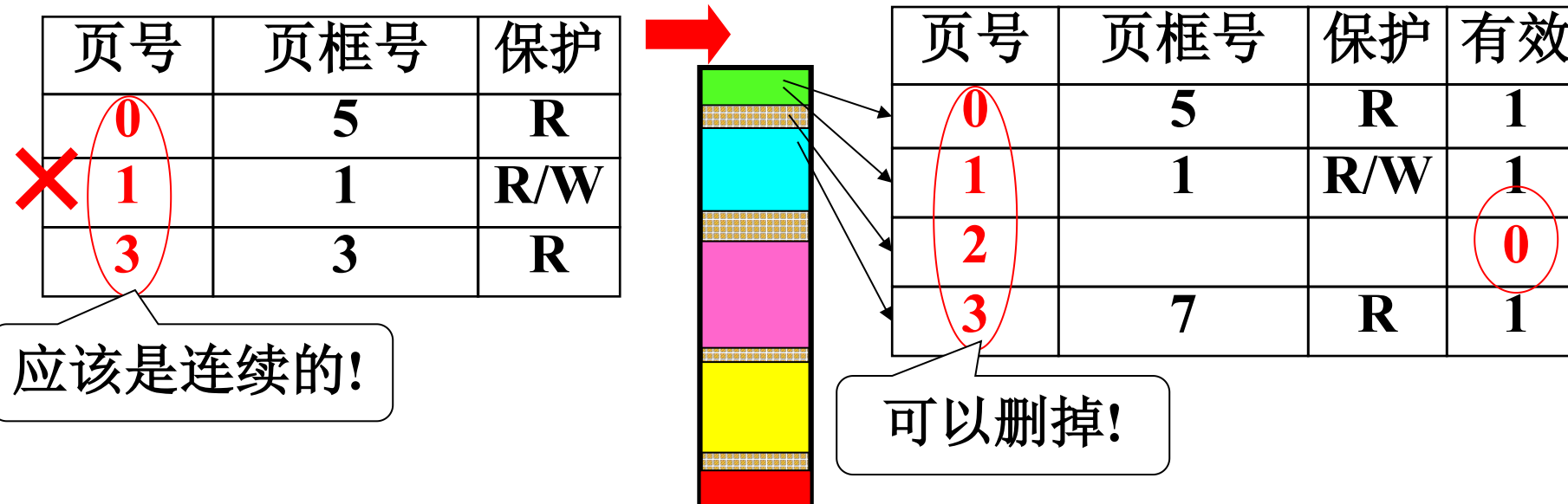
5	240
---	-----

物理地址: 0x5240

权限检查

来考虑一些细节问题

- 和段表不一样，页表可能会很大!
- 页是用来解决碎片问题的=>页面尺寸应尽量小
- 页面尺寸通常为4K，32位机器，有 2^{20} 个页面
- 每条指令都需要查几次页表，因此查表效率很重要
- 如果页号不连续，需要查找，折半 $\log_2(2^{20})=20$



页表项数据结构

可以删掉!

■ 页表基本数据结构Page Table Entry(PTE)

页号	页框号	保护	有效
0	5	R	1
1	1	R/W	1
2			0
3	7	R	1

Intel x86结构的PTE

页框号(物理页号)ppn	保留	0	L	D	A	PCD	PWT	U	W	P
31-12 ?	11-9	8	7	6	5	4	3	2	1	0

- 由逻辑地址va找到物理地址，只有一级页表，每页4k

```
unsigned translate(unsigned va, int wr) {  
    struct pte *pte = &page_table[va >> 12]; // 页号  
    if(!pte->valid || (wr && !pte->writeable))  
        throw address_fault;  
    return (pte->ppn << 12) | (va & 0xfff);  
}
```

分页内存管理，逻辑地址空间范围

页号	页框号	保护	有效
0	5	R	1
1	1	R/W	1
2			0
3	7	R	1

逻辑地址和物理地址分开，其范围？

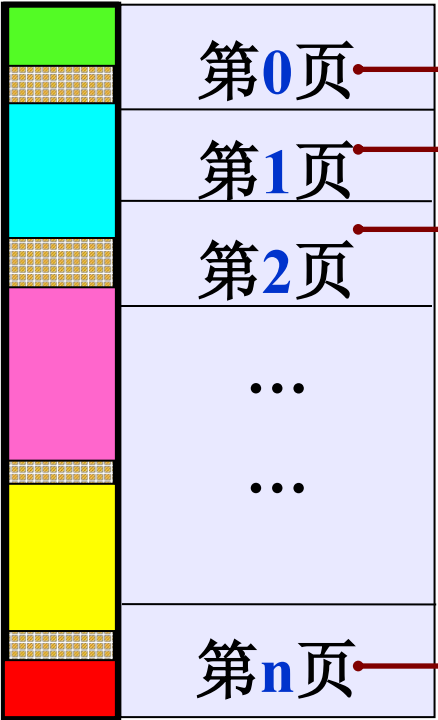
逻辑地址空间的页可以按需对应物理地址中的页框。
由虚到实

页框号 物理地址空间

用户程序逻辑地址空间

页表

页号 页框号



页号	页框号
0	2
1	7
2	4
...	...
...	...
n	m



分段内存

■ 32位地址空间+ 4K页面+页号连续 $\Rightarrow 2^{20}$ 个页表项

■ 2^{20} 个页表项, 每个4字节, 都放在内存, 要4M内存

■ 系统中并发10个进程, 需要40M内存

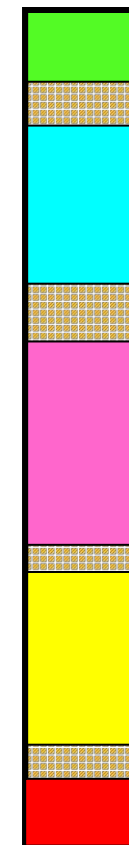
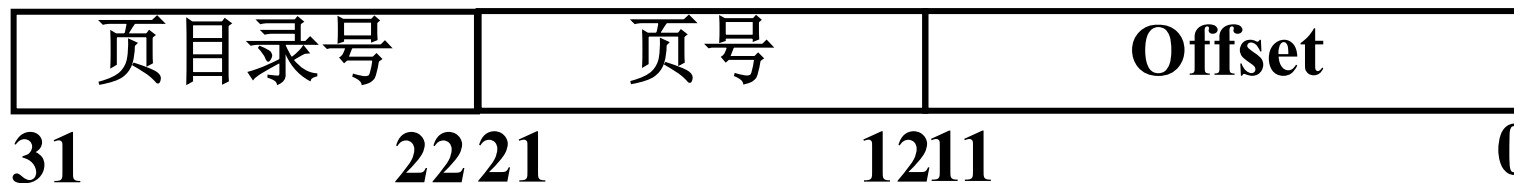
为什么每个进程需要独立一个页表?

■ 实际上大部分逻辑地址根本不会用到

32位: 总空间[0, 4G-1]!

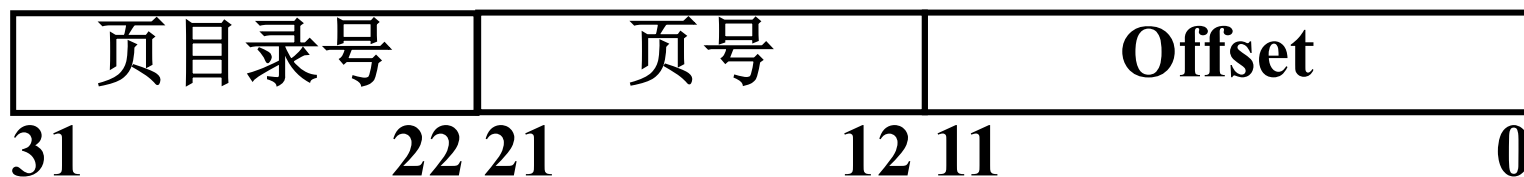
■ 引入多级页表, 顶层页表常驻内存, 不需要映射的逻辑地址不需要建立页表项

32位逻辑地址格式(多级页表)



- 32位地址，4G内存：先按照4M分割，每个4M内再按照4K分割。

32位逻辑地址格式(多级页表)

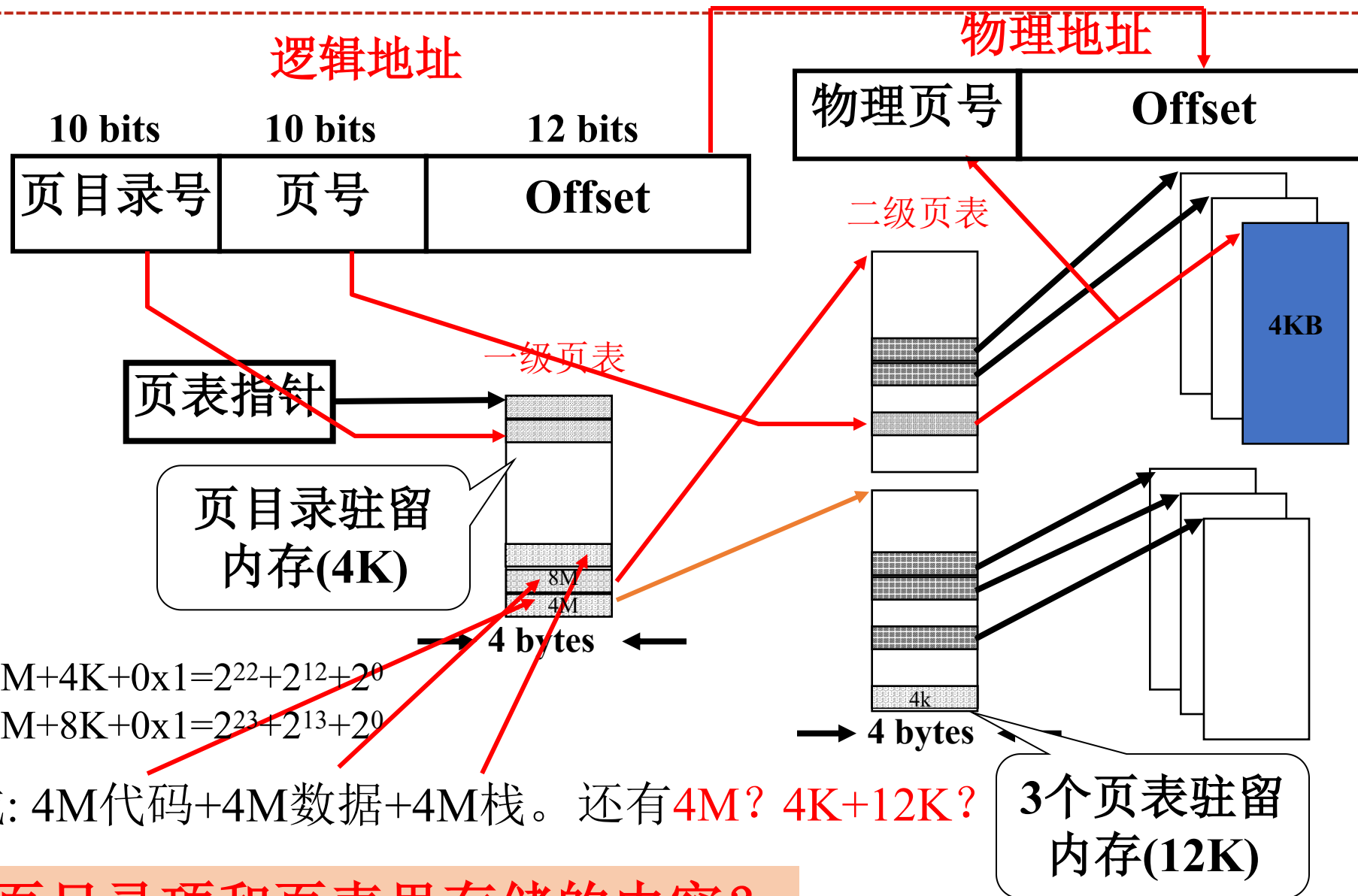


- 给定两个逻辑地址：

逻辑地址1: $4M+4K+0x1=2^{22}+2^{12}+2^0$

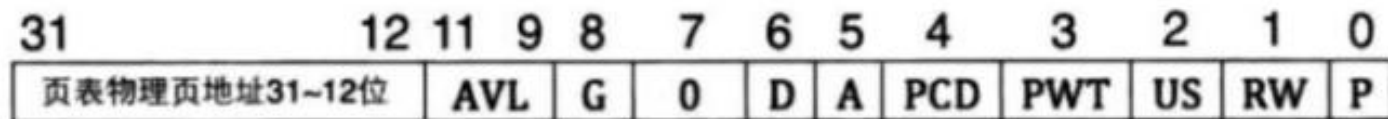
逻辑地址2: $8M+8K+0x1=2^{23}+2^{13}+2^0$

多级页表时的地址翻译

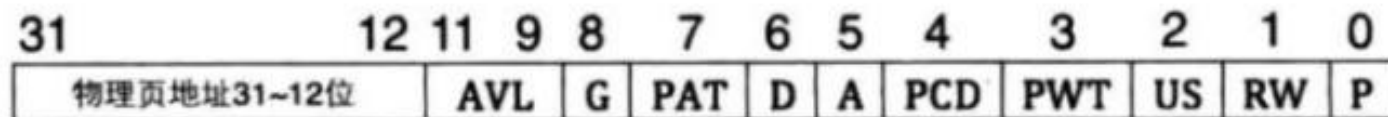


页目录项和页表里存储的内容?

PDE与PTE的结构



页目录项



页表项

P位：存在位，为1时表示该页在内存中

RW：读写位，为1时可读可写，为0是可读不可写

US：普通用户/超级用户位，为1时表示处于用户级，也就是3级特权级

PWT：页级通写位，为1表示此项采用通写方式，表示该页不仅是普通内存，还是高速缓存

PCD：页级高速缓存禁止位，为1表示该页启用高速缓存

A：访问位，为1表示该页被CPU访问过

D：脏页位，当CPU对一个页面执行写操作，此为被赋1

PAT：页属性表位，能够在页面一级的粒度上设置内存属性

G：全局位，为1表示该页在高速缓存TLB中一直保存

■ 一个32位的内存管理系统使用了两级页表，其逻辑地址中，第22到31位是第一级页表的索引，12位到21位是第二级页表的索引，页内偏移占0到11位。

- 1、请问进程整个地址空间有多少字节？一页有多少字节？
- 2、如果某个页目录号下对应的所有二级页表全部分配，共占用多少页面？一个二级页表的存储需要多少字节？
- 3、逻辑地址0xC030F000对应的页目录号和页号分别是多少？

10 bits	10 bits	12 bits
页目录号	页号	Offset

(注意B代表字节，一个32位地址占4字节)

多级页表使得地址翻译效率更低

一次
地址
访问

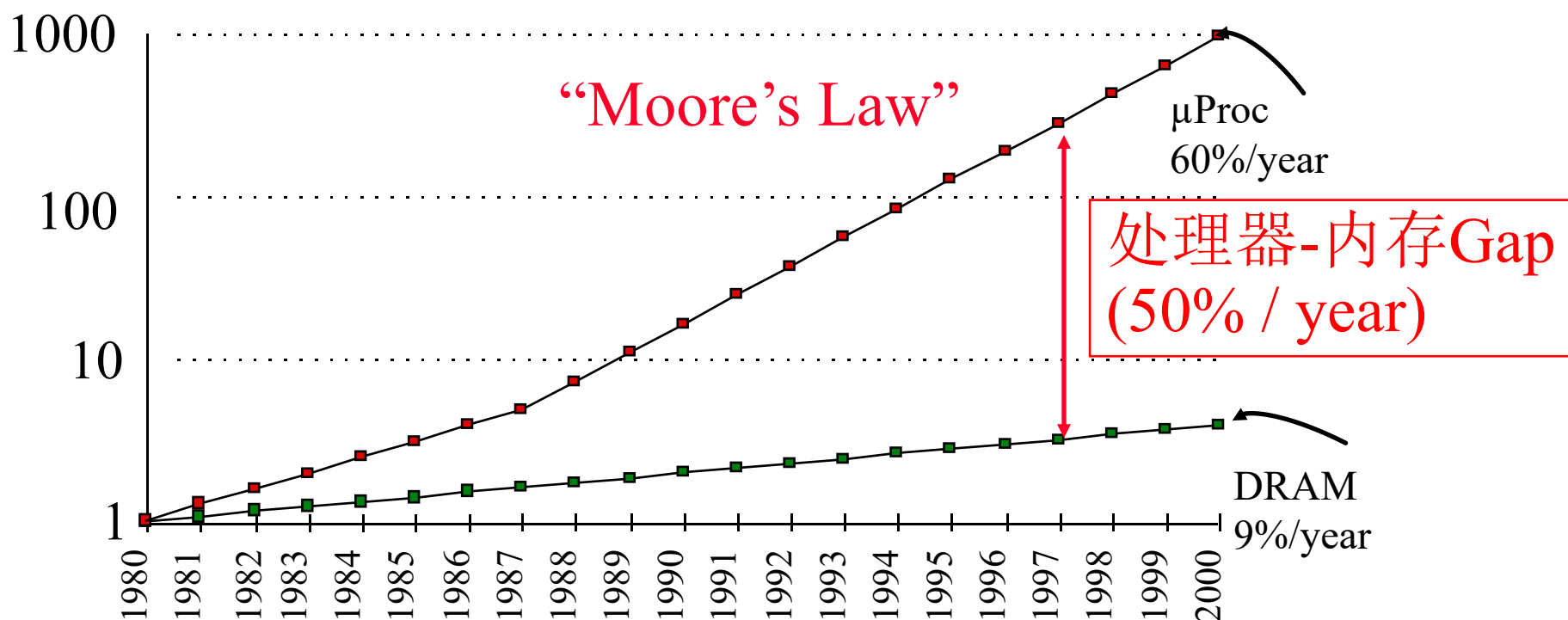
1级页表访存2次，速度下降50%

2级页表访存3次，速度下降到33%

3级页表访存4次，速度下降到25%

访问页表计算物理地址，
从物理地址取内容

需要注意的事实：内存
相比CPU本来就很慢！



- 多级页表的地址翻译效率很低，要提高效率
- 提高效率的基本想法: 硬件支持
- 要很快: 这个硬件放在哪里? 寄存器
- 页表小=>寄存器可行，但如果页表很大呢?
- TLB(Translation Look-aside Buffer)是一组关联快速寄存器组

有效	页号	修改	保护	页框号
1	140	0	R	56
1	20	1	R/W	23
0	19	0	R/X	29
1	21	0	R	43

可参考《步步惊心——
软核处理器内部设计分析》

逻辑地址

页号	Offset
----	--------

有效	页号	修改	保护	页框号
1	140	0	R	56
1	20	1	R/W	23
0	19	0	R/X	29
1	21	0	R	43

物理地址

物理页号	Offset
------	--------

TLB命中

关联查找-
同时进行!

TLB

页表

TLB未命中(失
效)

还要查页表,
似乎更慢了!

TLB得以发挥作用分析

■ TLB命中时效率会很高，未命中效率会降低，平均后仍表现良好。用数字来说明：

$$\text{有效访问时间} = \text{HitR} \times (\text{TLB} + \text{MA}) + (1 - \text{HitR}) \times (\text{TLB} + 2\text{MA})$$

命中率!

内存访问时间!
假设100ns

TLB时间!
假设20ns

$$\text{有效访问时间} = 80\% \times (20\text{ns} + 100\text{ns}) + 20\% \times (20\text{ns} + 200\text{ns}) = 140\text{ns}$$

$$\text{有效访问时间} = 98\% \times (20\text{ns} + 100\text{ns}) + 2\% \times (20\text{ns} + 200\text{ns}) = 122\text{ns}$$

TLB要想发挥作用，命中率应尽量高

TLB越大越好，但TLB价格昂贵，通常条目数在[64, 1024]

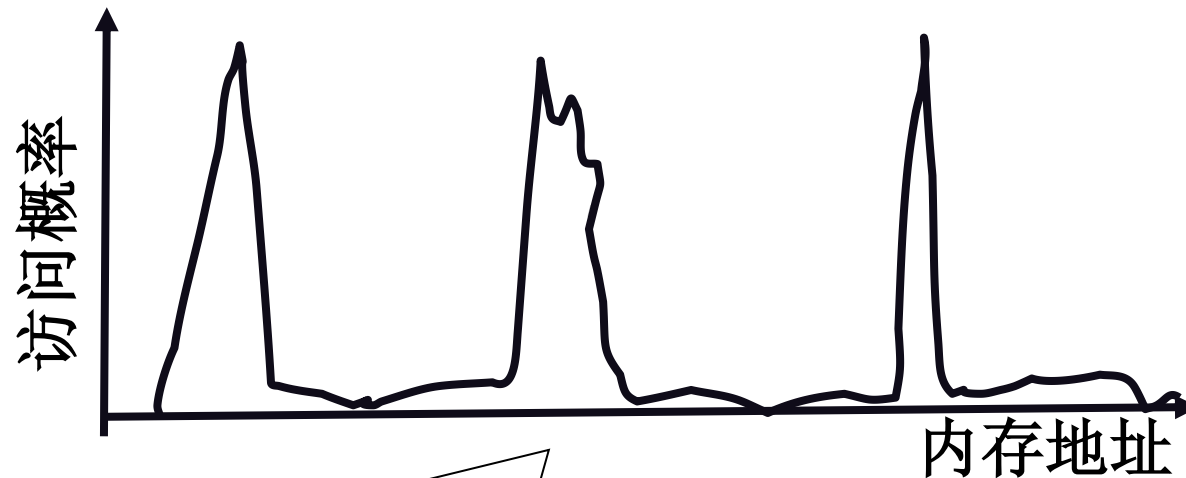
平均加快了
13%!

为什么TLB条目数在64-1024之间?

- 相比 2^{20} 个页, 64很小, 为什么TLB就能起作用?
- 程序的地址访问存在局部性

➤ 空间局部性(Locality in Space)

➤ 时间局部性



程序多体现为循环、顺序结构



局部性又是计算机的一个基本特征

TLB条目少，页表项多=>TLB动态变化

- 如果TLB未命中，可将查到的页表项载入TLB
- 如果TLB已经满了，需要选择一个条目来替换
- 有些时候希望某些条目固定下来（如内核代码），某些TLB的设计有这样的功能，不被选择替换
- 进程切换后，所有的TLB表项都变为无效（flush）
- 如果进程马上又切换回来，则这种策略就很低效。有的TLB设计中条目项保存ASID（Address-space identifier），此时不需要flush。

■ 例1：已知某分页系统，主存容量为64k，页面大小为1k，对一个4页大的作业，第0、1、2、3页被分配到内存的2、4、6、7块中。求：将十进制的逻辑地址1023、2500、4500转换成物理地址。

解：

(1) $1023/1K$ ，得到页号为0，页内地址1023。

又 对应的物理块号为2，故物理地址为 $2*1k+1023=3071$

(2) $2500/1K$ ，得到页号为2，页内地址452

又 对应的物理块号为6，故物理地址为 $6*1k+452=6596$

(3) $4500/1K$ ，得到页号为4，页内地址404

因为页号不小于页表长度，故产生越界中断

■ 设有8页的逻辑空间，每页有1024字节，它们被映射到32块的物理存储区中，那么逻辑地址的有效位是__位，物理地址至少是__位。

分析

- 逻辑地址有两个部分组成：页号和页内偏移地址。逻辑空间有8 (2^3) 页，说明页号需要3个二进制位描述，而每页有1024 (2^{10}) 字节，说明页内偏移地址为10二进制位描述，因此逻辑地址的有效位为 $3+10=13$ 位。
- 因为物理地址与逻辑地址的页面大小相同，而物理存储块为32 (2^5) 占5位，所以物理地址至少为 $5+10=15$ 位

■ 实现机理

- 逻辑地址空间和内存都分割大小相等的片（页和页框）
- 每个进程用页表（多级、反向等）建立页和页框的映射
- 进程创建时申请页，可用表、位图等结构管理空闲页
- 逻辑地址通过页表算出物理地址，到达内存
- 进程切换时，页表跟着切换

分页更适合于自动化（硬件实现）！

■ 优点：靠近硬件，结构严格，高效使用内存

■ 缺点：不符合程序员思考习惯

■ 1 背景

- (1) 程序的装入、运行和地址映射;
- (2) 逻辑地址空间和物理地址空间
- (3) 进程内外存交换

■ 2 连续内存分配

- (1) 固定等长分区、固定变长分区、可变分区、分区分配算法;
- (2) 碎片问题与内存紧缩

■ 3 分段内存管理

■ 4 分页内存管理

■ 5 段页结合内存管理

段、页结合!

让段面向用户、让页面向硬件



思考：翻译逻辑地址
0xc2f0、0x0240？

段号+偏移(cs:ip)

逻辑地址

段号	基址	长度	保护
0	0x4000	0x0800	R
1	0x4800	0x1400	R/W
2	0xF000	0x1000	R/W
3	0x0000	0x3000	R



页号

偏移

物理地址

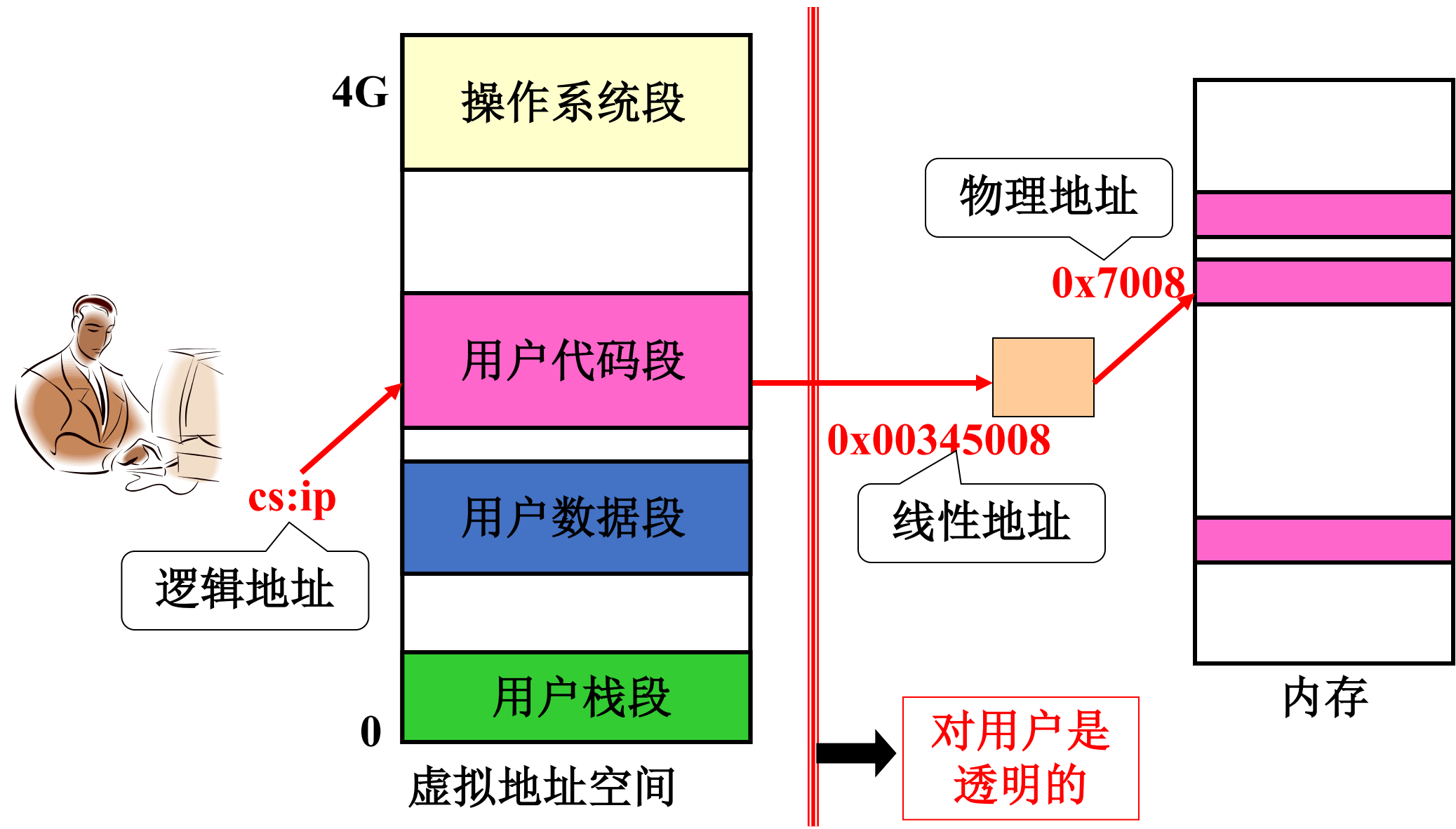
物理页号

偏移

页框号	保护
5	R
1	R/W
3	R/W
7	R

常称为线性地址，以示区别

段页式内存管理的基本视图



■ 实现机理

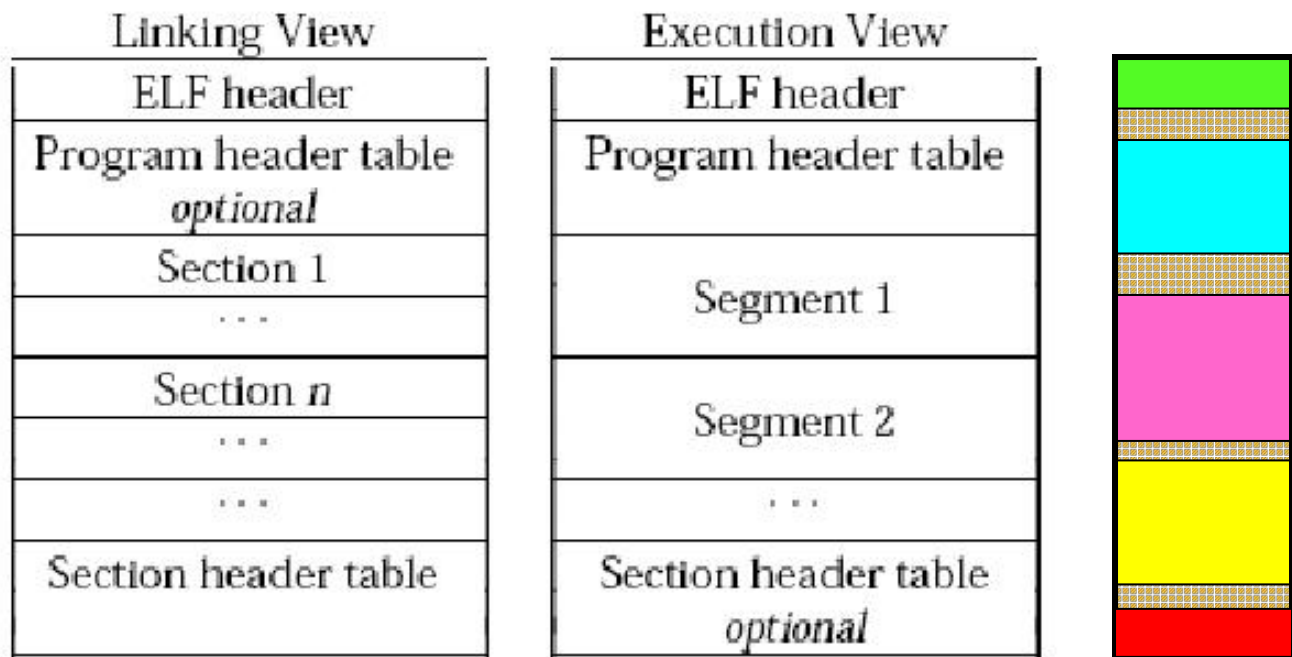
- 程序的段划分的是线性地址空间（如0-4G）
- 线性地址空间和内存被分割大小相等的片（页和页框）
- 进程用页表建立页和页框的映射
- 进程创建申请段（线性地址空间），段申请页（物理内存）
- 逻辑地址通过段表加页表算出物理地址，到达内存
- 进程切换时，段表和页表都跟着切换

■ 优点：符合程序员习惯，并可高效利用内存

■ 缺点：复杂，访问一次地址需要查表好多次...

操作系统的可执行文件格式

- 每种操作系统都规定了可以运行文件格式（可执行文件）：
 - COFF（Common Object File Format 通用对象文件格式） **Unix**
 - ELF（Executable and Linking Format 可执行和链接格式） **Linux**
 - PE（Portable Executable） **Windows**





Linux下可执行文件格式

■ 一个ELF文件中重要的段包括:

- .text 段: 存储只读程序
- .data 段: 存储已经初始化的全局变量和静态变量
- .bss 段: 存储未初始化的全局变量和静态变量, 因为这些变量的值为0, 所以这个段在文件当中不占据空间
- .rodata 段: 存储只读数据, 比如字符串常量

```
int printf(const char *format, ... );
int global_init_var = 16;
int global_unint_var;
void func1 (int );
int main()
{
    static int static_var = -32;
    static int static_var_uninit;

    int a = 1;
    int b;

    func1(static_var + global_init_var + a + b);

    return a;
}

void func1 (int i)
{
    printf("%d\n", i);
}
```

```
[root@centos]# readelf -h SimpleSection.o
ELF Header:
  Magic:      7f 45 4c 46 01 01 01 00 00 00 00 00 00 00 00 00
  Class:                                ELF32
  Data:                                      2's complement, little
  endian
  Version:                                1 (current)
  OS/ABI:                                  UNIX - System V
  ABI Version:                            0
  Type:                                    REL (Relocatable file)
  Machine:                                  Intel 80386
  Version:                                  0x1
  Entry point address:                    0x0
  Start of program headers:               0 (bytes into file)
  Start of section headers:              224 (bytes into file)  0xe0
  Flags:                                  0x0
  Size of this header:                    52 (bytes)  0x34
  Size of program headers:                0 (bytes)
  Number of program headers:              0
  Size of section headers:                40 (bytes)
  Number of section headers:              11
  Section header string table index:      8
```

```
[root@centos]# readelf -S SimpleSection.o
There are 11 section headers, starting at offset 0xe0:
```

段序号、段名称、段类型、段虚拟地址、偏移量、大小、ES、标志、Lk、Inf、对齐

Section Headers:

[Nr]	Name	Type	Addr	Off	Size	ES	Flg	Lk	Inf	Al
[0]		NULL	00000000	000000	000000	00		0	0	0
[1]	.text	PROGBITS	00000000	000034	000020	00	AX	0	0	4
[2]	.rel.text	REL	00000000	0003f4	000010	08		9	1	4
[3]	.data	PROGBITS	00000000	000054	000008	00	WA	0	0	4
[4]	.bss	NOBITS	00000000	00005c	000004	00	WA	0	0	4
[5]	.rodata	PROGBITS	00000000	00005c	000004	00	A	0	0	1
[6]	.comment	PROGBITS	00000000	000060	00002d	01	MS	0	0	1
[7]	.note.GNU-stack	PROGBITS	00000000	00008d	000000	00		0	0	1
[8]	.shstrtab	STRTAB	00000000	00008d	000051	00		0	0	1
[9]	.symtab	SYMTAB	00000000	000298	0000f0	10		10	10	4
[10]	.strtab	STRTAB	00000000	000388	00006b	00		0	0	1

Key to Flags:

W (write), A (alloc), X (execute), M (merge), S (strings)
I (info)
O (extra)

```
[root@centos]# ls -al SimpleSection.o
-rw-r--r-- 1 root root 1028 Aug 21 10:05 SimpleSection.o
```

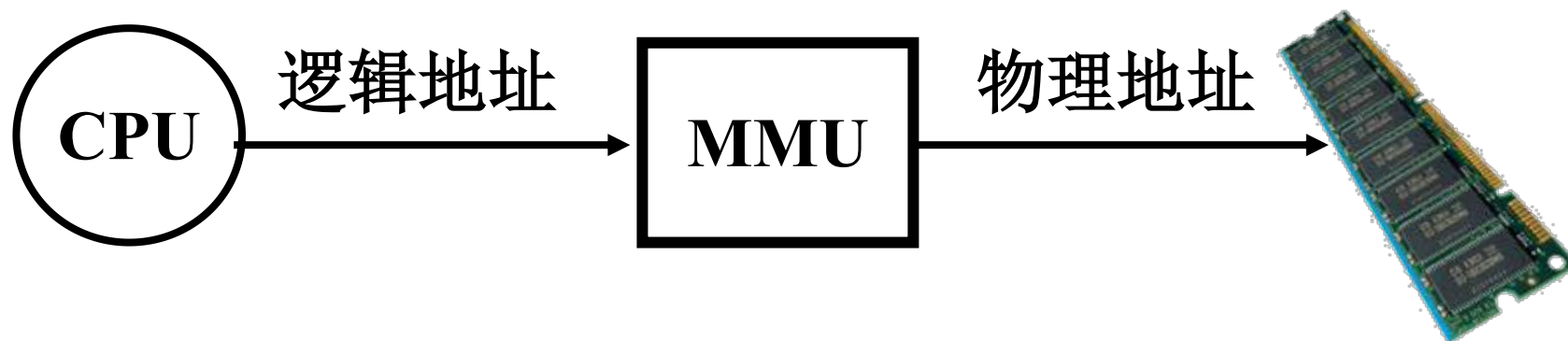
试试能不能算出总大小?

Intel x86的内存

■ 段页结合时，进行一次地址翻译需要：

(1)找到段表；(2)查段表；(3)找TLB；(4)找到页目录表；(5)查找页目录项；
(6)找到页表；(7)查找页表；(8)形成物理地址；(9)需要段越界检查；(10)需要进行段保护权限检查；(11)需要进行页保护权限检查...

■ 如此多的事情都用软件实现，其效率会很低



- 实模式，内存地址20位，1M。
- 保护模式，地址32位，4G。
- CR0.PE位=1进入实模式。

■ 实模式下: (CS, DS, ES, SS)

■ 逻辑地址与物理地址一致

```
mov cx, 0x2000
```

```
mov ds, cx    #基地址0x20000 (左移4位, 4字节对齐)
```

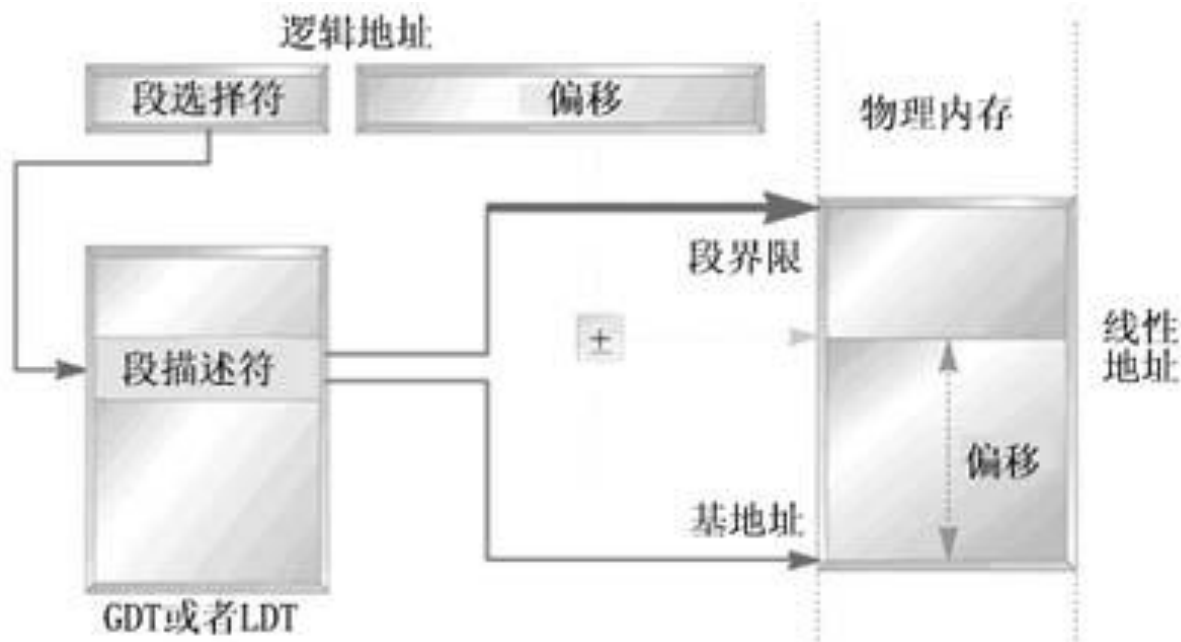
```
mov [0xc0], ax    # 实际写入物理地址0x200c0
```


实模式地址与保护模式地址

■ 实模式：地址空间1MB



■ 保护模式：地址空间4GB



X86地址变换

- 保护模式开启分页机制;
- 实模式和保护模式分段管理有区别:
 - 实模式段寄存器中存储段基址;
 - 保护模式段选择器存储段描述符表中的索引;

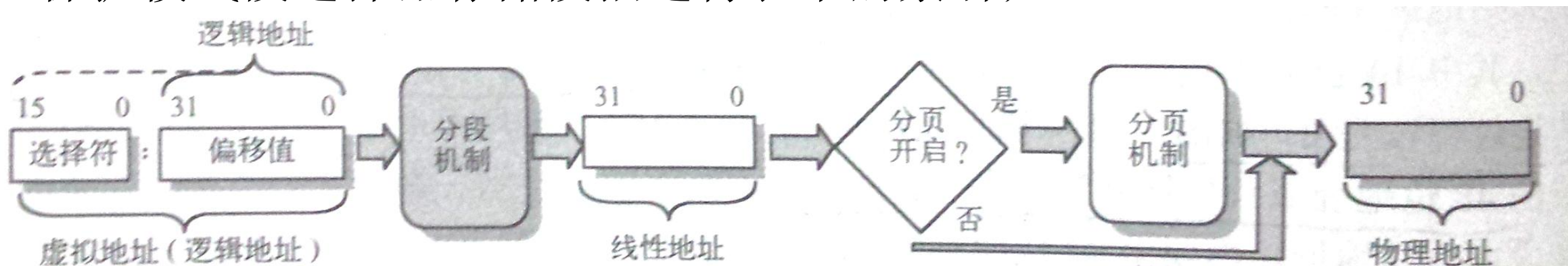
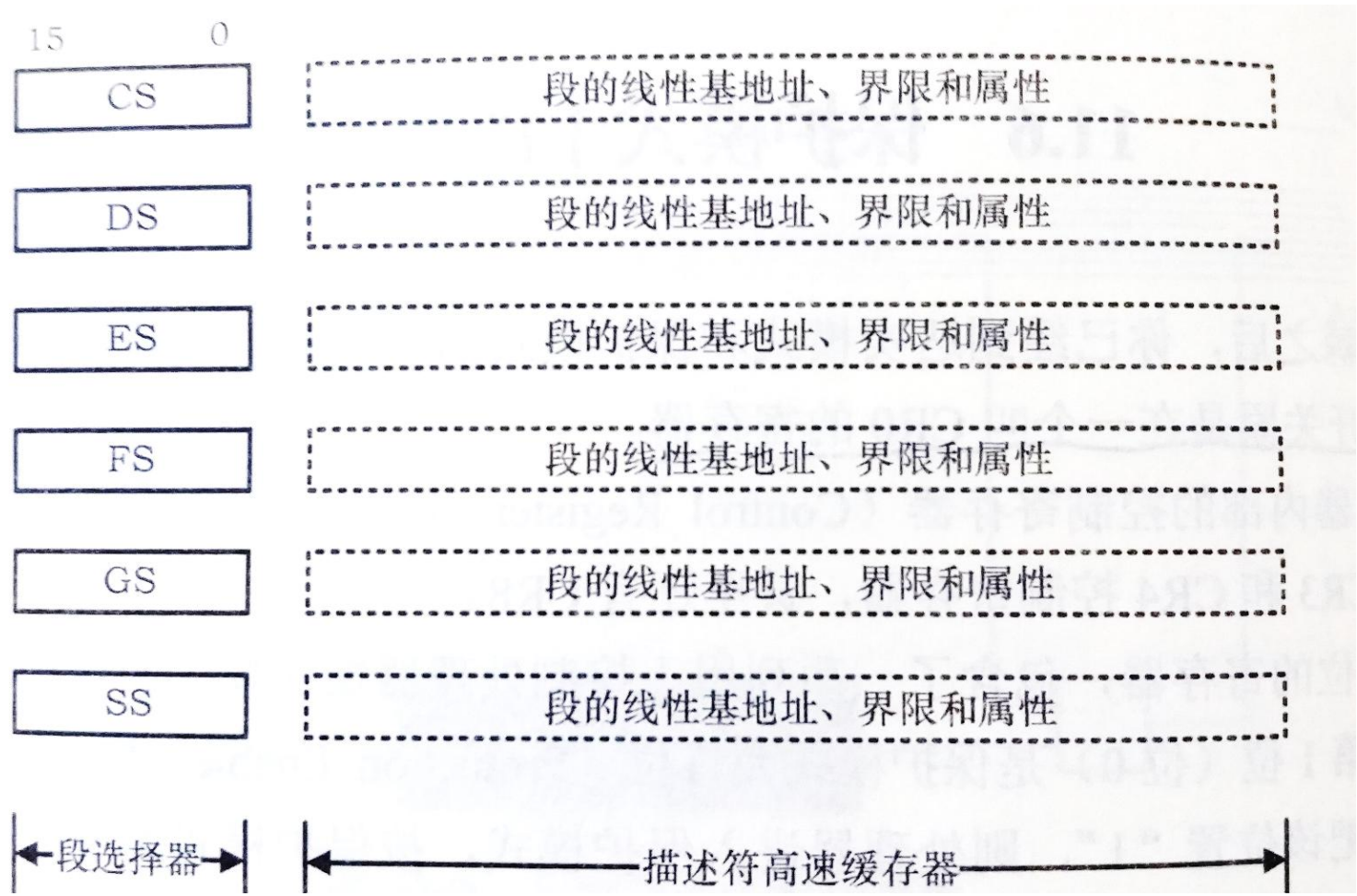


图 4-4 虚拟地址（逻辑地址）到物理地址的变换过程

嵌入式软件多数运行在没有（未开启）分页机制的处理器上

- 实模式称为：段寄存器(CS, DS, ES, SS)
- 保护模式称为：段选择器（6个）



■ 选择符：CS, SS, DS, ES, FS, GS（16位寄存器）

■ 偏移值：EIP, ESP, ESI, EDI（32位寄存器）

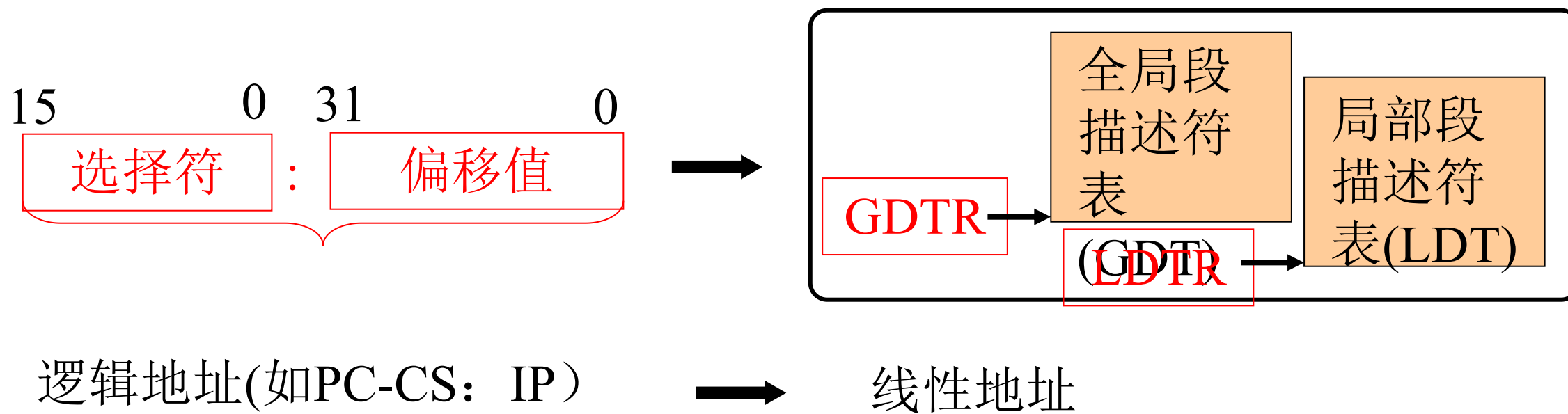
段号	基址	长度	保护
0	0x4000	0x0800	R
1	0x4800	0x1400	R/W
2	0xF000	0x1000	R/W
3	0x0000	0x3000	R

■ GDT：全局段表（进程共享，OS各段，进程LDT/TSS入口）

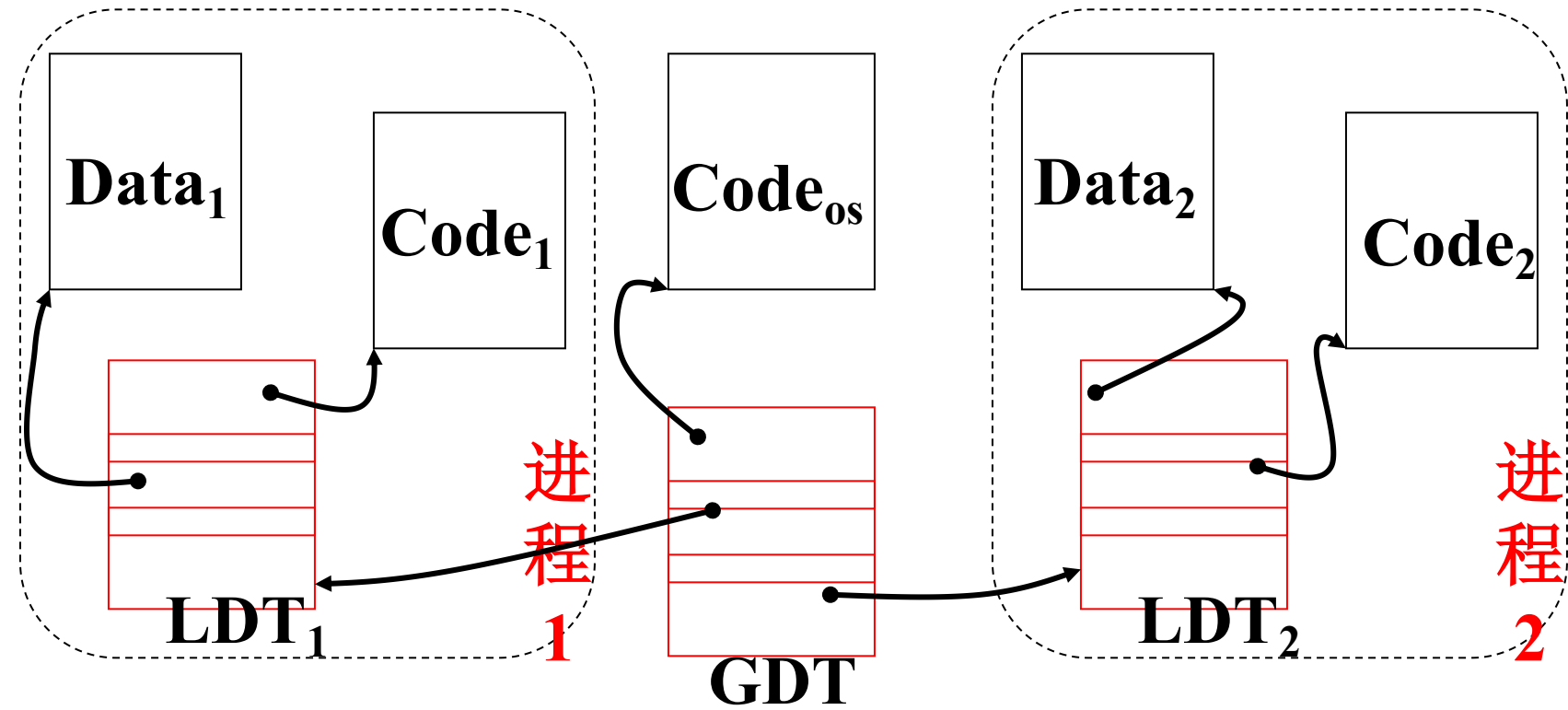
■ LDT：局部段表（各进程独有，描述进程各段）

■ GDTR和LDTR：32位段表基址（线性地址）+16位段表长度。需用特权指令LLDT和SLDT等。

■ TSS: 任务状态段，保存任务各寄存器值。通过TR寄存器访问



- 每个进程需要有自己的段表
- 操作系统有自己的段表，同时还要记录访问每个进程的段表入口

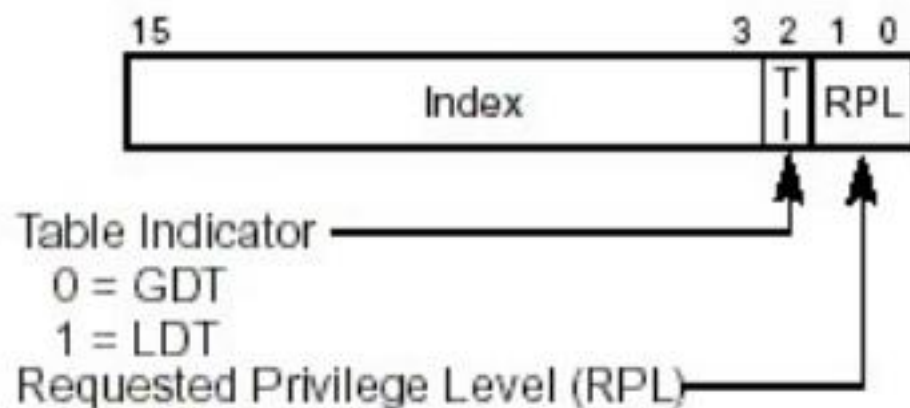


段描述符:
LDT(GDT)
中的表项

64	55	47	39	32
段基地址 (BASE) 31..24	G D 0 AV L	限长 19..16	P DPL 1 type	段基地址 (BASE) 23..16
段基地址 (BASE) 15..0			段限长 (LIMIT) 15..0	
31	23	15	7	0

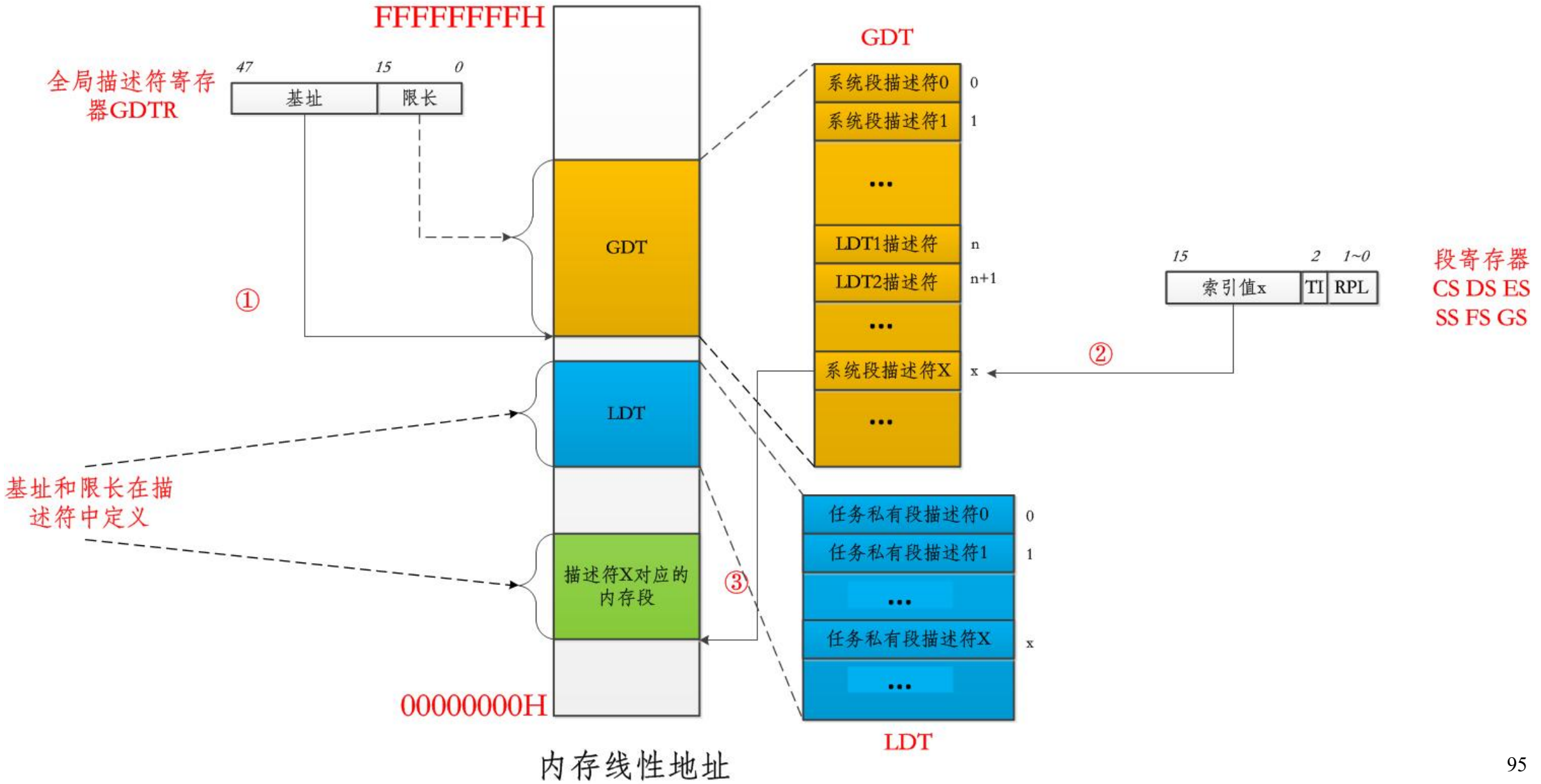
■ 当TI=0时表示段描述符在GDT中

- ① 先从GDTR寄存器中获得GDT基址。
- ② 然后再GDT中以段选择器高13位位置索引值得到段描述符。



- ③ 段描述符包含段的基址、限长、优先级等各种属性，这就得到了段的起始地址（基址），再以基址加上偏移地址得到线性地址。

Intel x86的分段硬件--LDT、GDT



■ 当TI=1时表示段描述符在LDT中

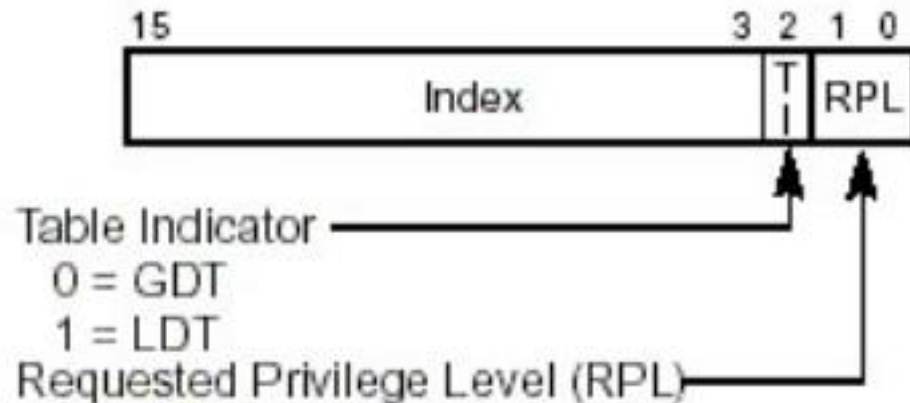
➤ ① 还是先从GDTR寄存器中获得GDT基址

➤ ② 从LDTR寄存器中获取LDT所在段的位
置索引(LDTR高13位)

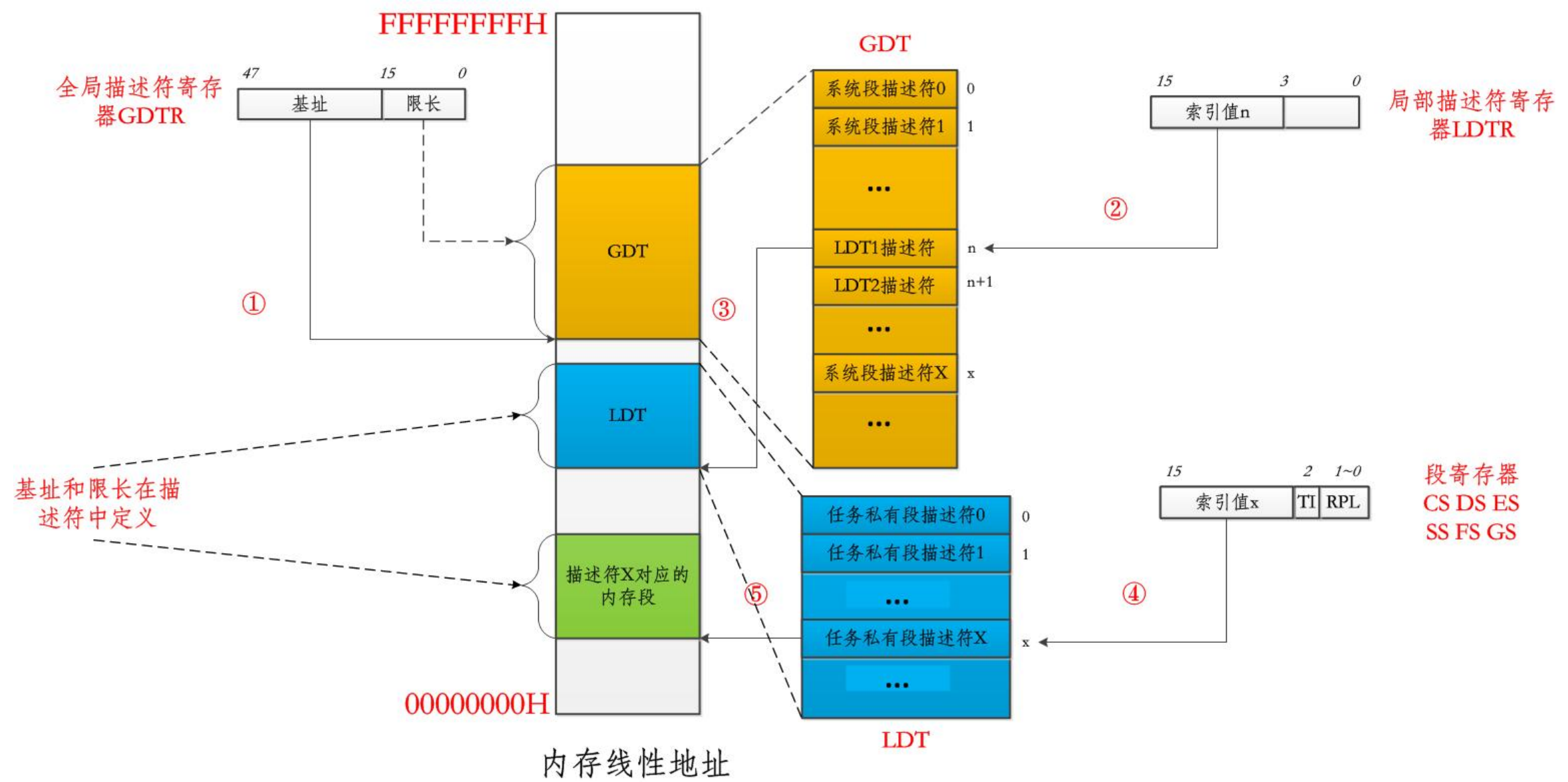
➤ ③ 以这个位置索引在GDT中得到LDT段描述符从而得到LDT段基址

➤ ④ 用段选择器高13位位置索引值从LDT段中得到段描述符

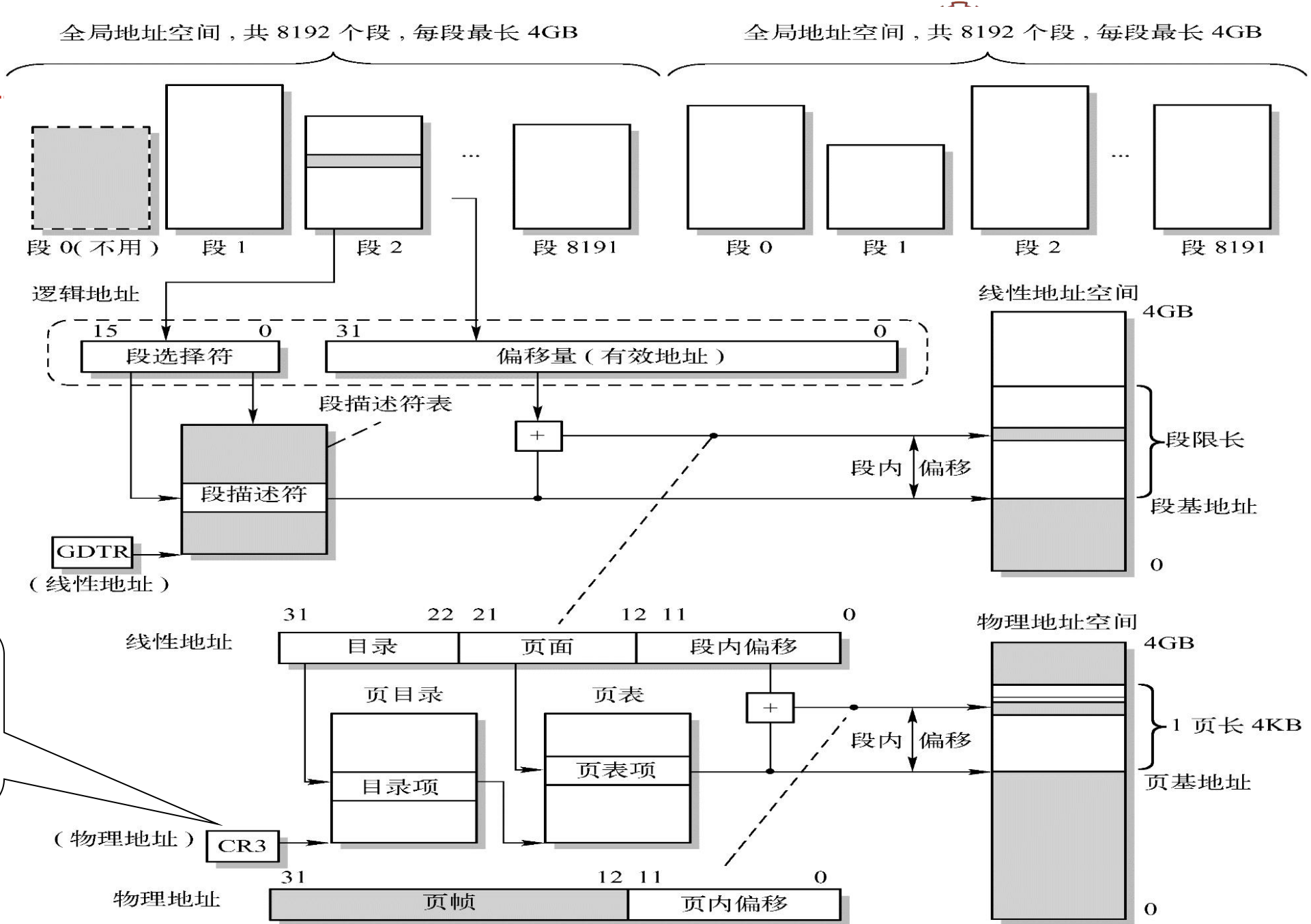
➤ ⑤ 段描述符包含段的基址、限长、优先级等各种属性，这就得到了段的起始地址（基址），再以基址加上偏移地址得到线性地址



Intel x86的分段硬件 -- LDT、GDT

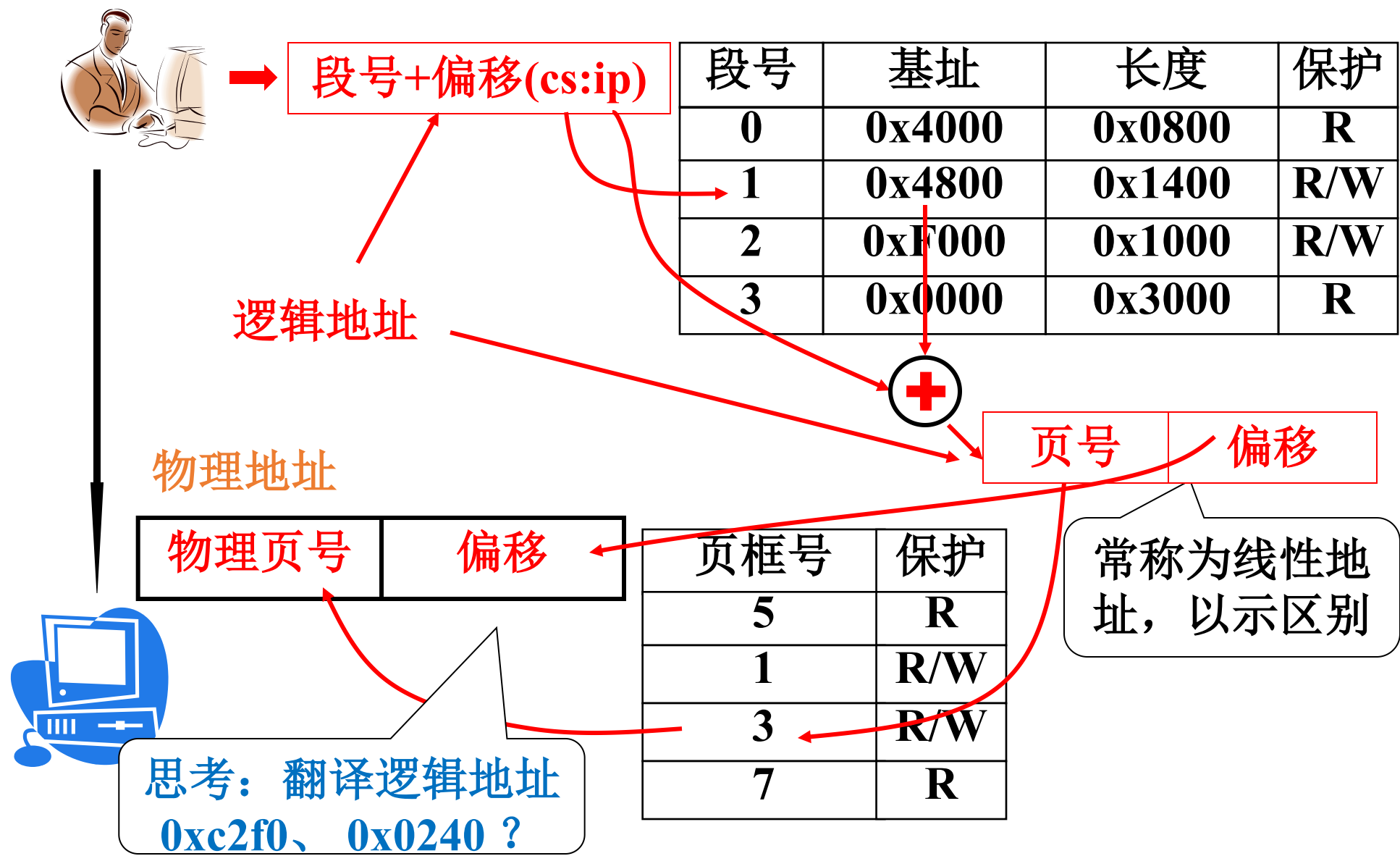


X86地址变换

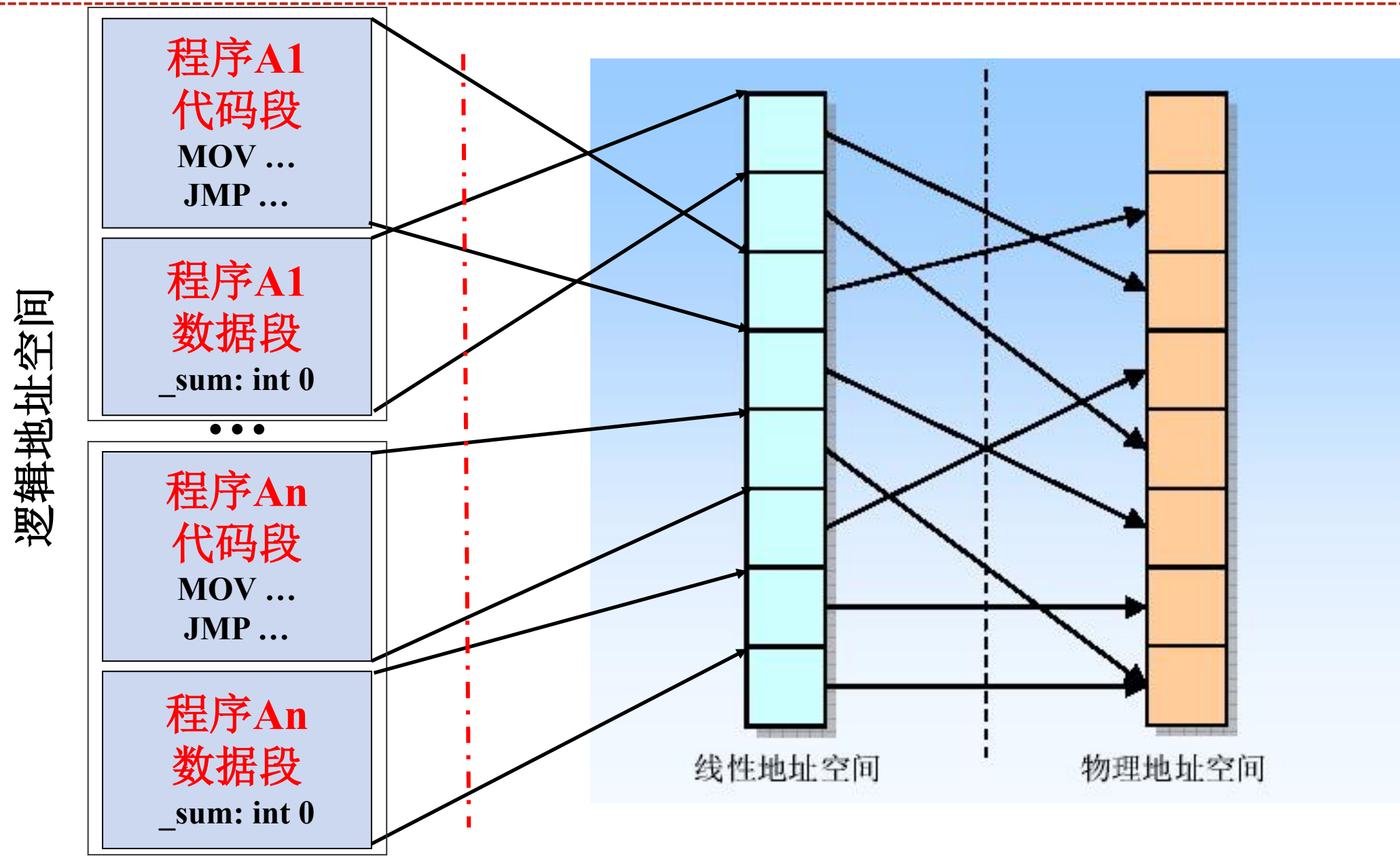


多个进程的段基址是否可以重叠

- 内存的根本目的 $\Rightarrow$ 把程序放在内存并让其执行
- 程序执行需要重定位 $\Rightarrow$ 编译、载入和运行三种定位时刻
- 运行时重定位最成熟 $\Rightarrow$ 从逻辑地址到物理地址的翻译
- 内存如何管理 $\Rightarrow$ 连续内存分配（分区）最直观
- 程序由若干段组成 $\Rightarrow$ 以段为单位的内存分区策略 $\Rightarrow$ 分段
- 分段对程序员自然，但会造成内存碎片 $\Rightarrow$ 分页 $\Rightarrow$ 段页结合
- 翻译、保护、内存分配是内存管理的三个核心词!

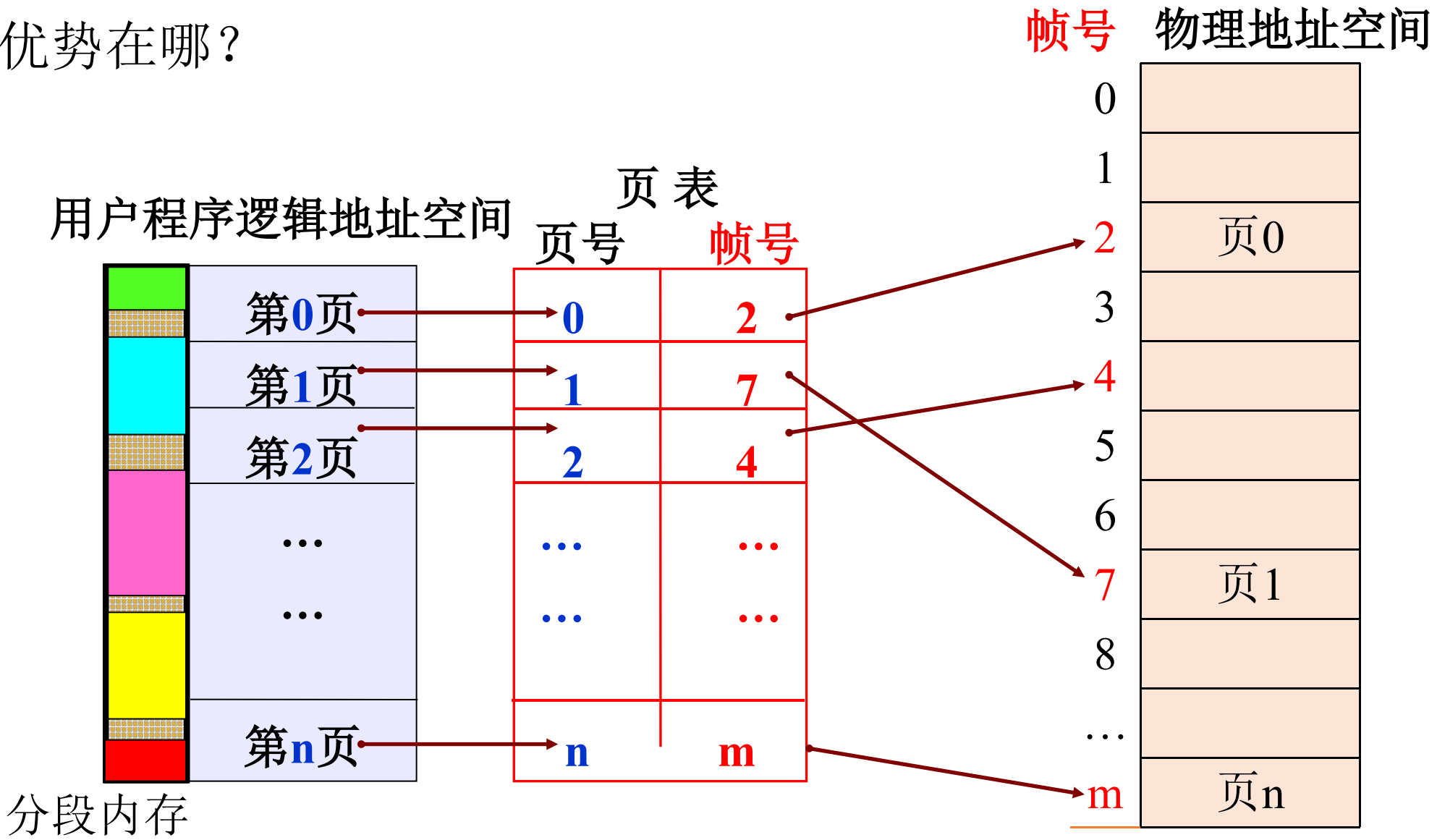


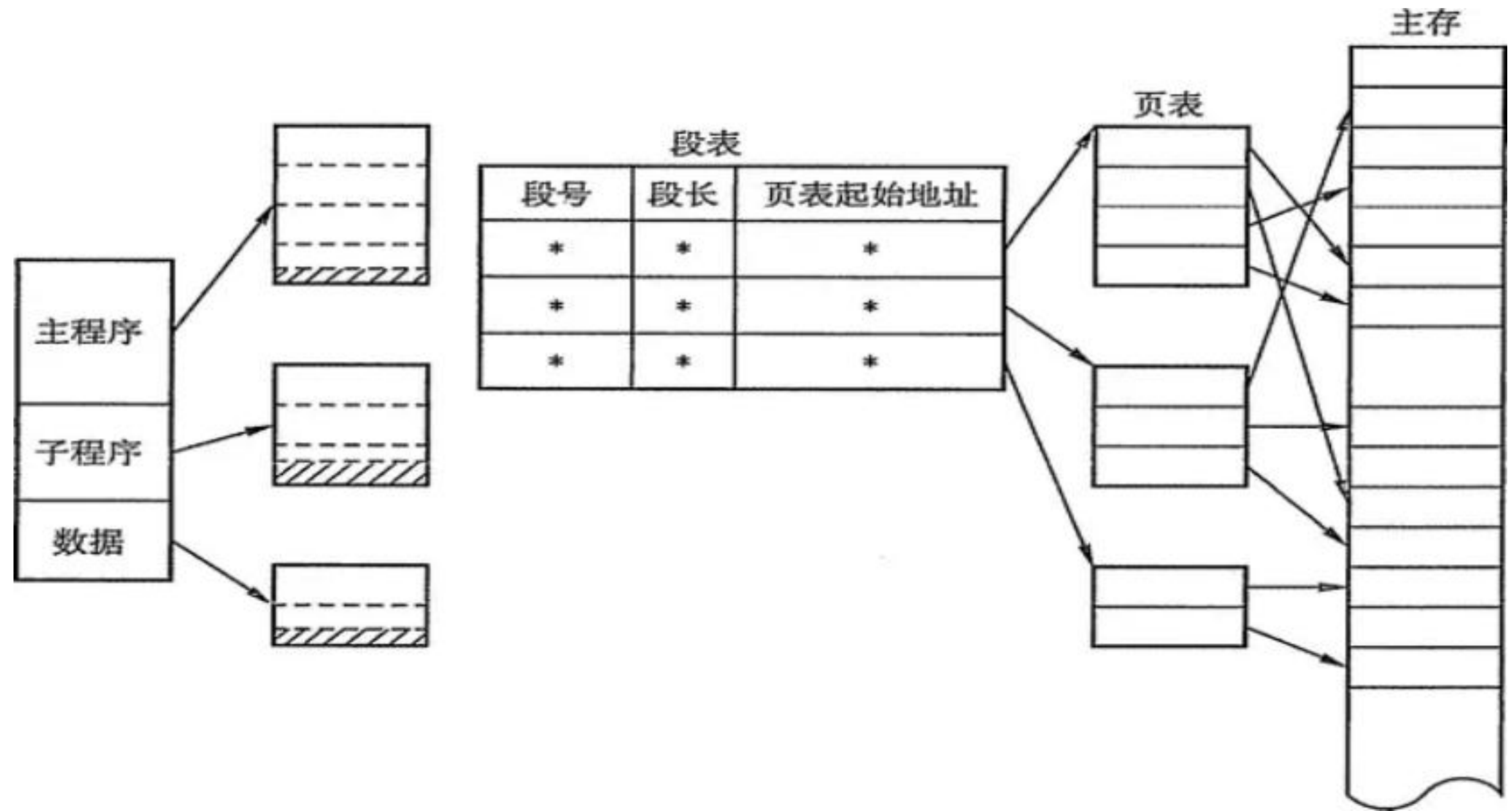
逻辑地址空间-线性地址空间-物理地址空间的关系



随堂讨论： 2)描述一下段页结合优势

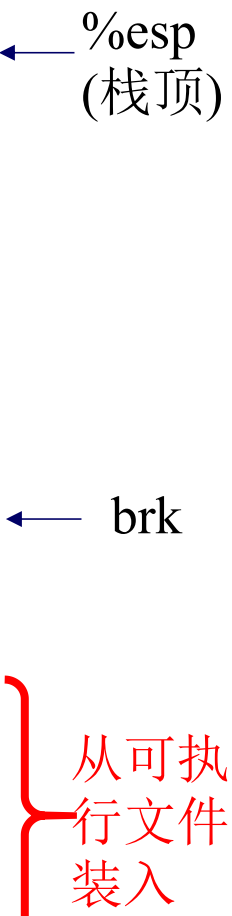
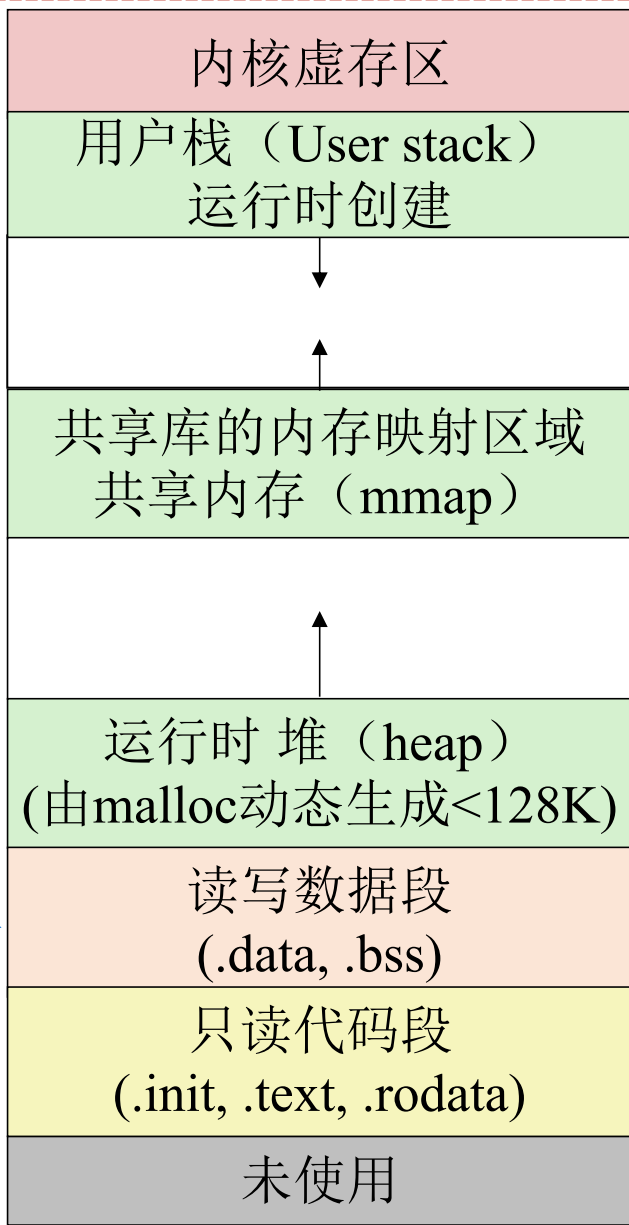
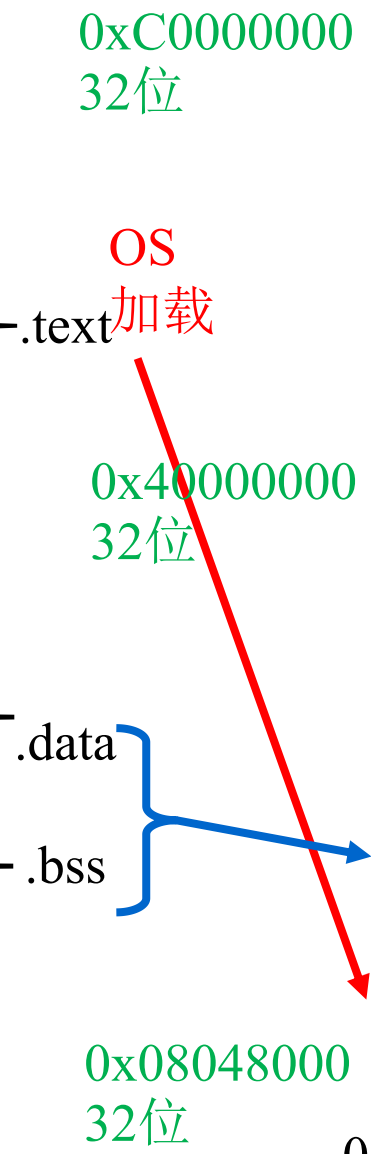
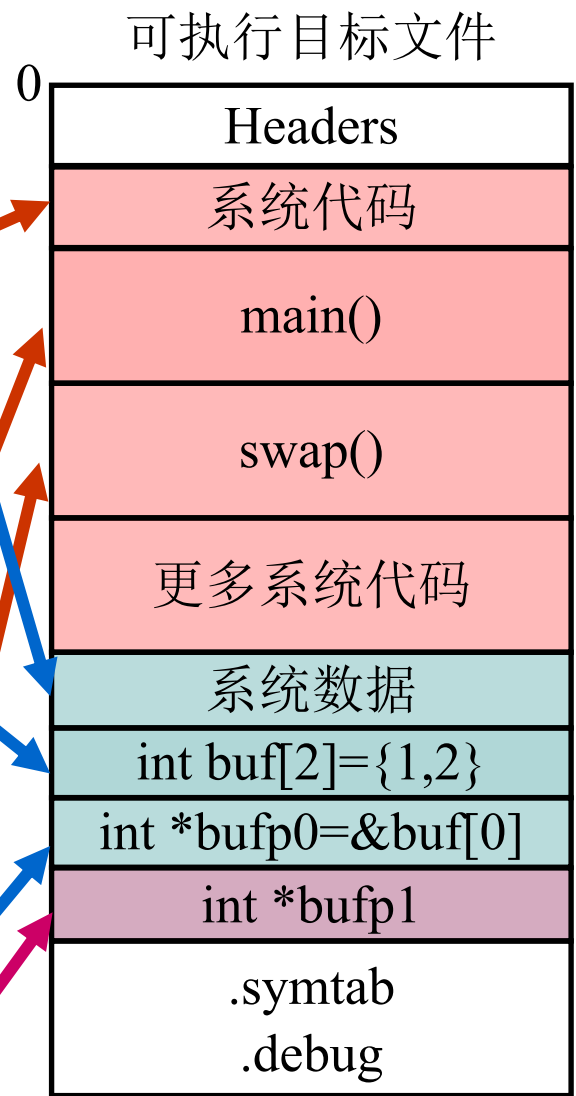
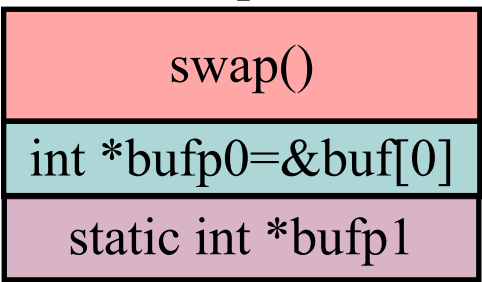
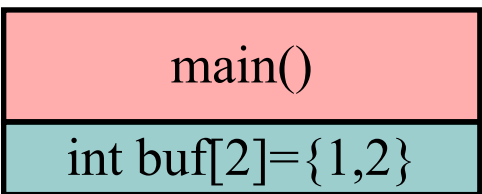
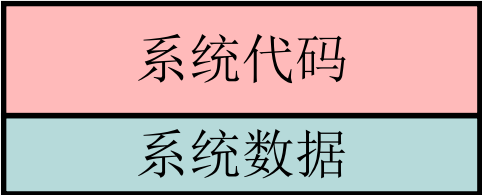
■ 如此映射优势在哪？





编译、链接

可重定位目标文件



Hope you enjoyed the OS course!